**TRANSPORT AND TELECOMMUNICATION INSTITUTE**




Bachelor of Natural Sciences in Computer Science

# Software Engineering Course Project

## "Application development for calculating and visualizing the distributed height of layers during 3D printing using the WAAM method."

Name Surname: Igors Volohovs

Group: 4202BDA

St. code.: St81588

Riga 2025

**ABBREVIATIONS** ..... **4**
**INTRODUCTION** ..... **5**
**PROJECT TASK** ..... **6**
Purpose of the work ..... 6
The task for the coursework ..... 6
**PROJECT ANALYSIS** ..... **7**
Specification ..... 7
Functional requirements ..... 7
Non-functional requirements ..... 7
System Users ..... 8
**INCEPTION PHASE** ..... **8**
Business requirements ..... 8
User Stories: As a … , I want … , so that … ..... 8
Acceptance criterias: Given … , when … , then … ..... 9
**ELABORATION PHASE** ..... **10**
**Use Case Diagram** ..... **10**
Actor: ..... 10
Use Case Elements: ..... 10
Scenarios for Use Case Elements ..... 11
Point cloud plot monitoring ..... 11
Current height distribution monitoring ..... 12
Delta heights distribution monitoring ..... 14
Choose monitoring path ..... 16
Class Diagram ..... 18
Description of classes ..... 19
Quality estimation ..... 19
Quality Estimation ..... 20
System metrics ..... 20
**Component diagram** ..... **21**
**CONSTRUCTION PHASE** ..... **22**
The 1st Iteration ..... 22
The 2st Iteration ..... 23
The 3st Iteration ..... 25
Summary ..... 27
**CONCLUSIONS** ..... **27**
**REFERENCES** ..... **29**
**ANNEXE** ..... **30**
Screenshots ..... 30
Program code ..... 31
calc/calc.jl ..... 31
calc/config.jl ..... 31
calc/height.jl ..... 34

calc/plane.jl..... 45
calc/points.jl..... 48
plot/plot.jl..... 64
hlc.jl..... 69
main.jl..... 74
config.yaml..... 74

## ABBREVIATIONS

WAAM - Wire Arc Additive Manufacturing

HLC - Height Layer Calculator

WMC - Weighted Method Per Class

DIT - Depth of Inheritance Tree

NOC - Number of Children

NOM - Number of Methods

NC - Number of Components

CBO - Coupling Between Object classes

RFC - Response for a Class

LCOM - Lack of Cohesion in Methods

LOC - Line of Code

# INTRODUCTION

Every manufacturing operation strives to optimize its production process to save costs. The Additive Lab laboratory is researching new strategies for optimized metal 3D printing using arc welding. This printing (or welding) process involves an industrial robot and a human operator who controls it using a control panel.

Currently, to ensure high-quality printing, not all operations are performed automatically. This means that during the printing process, the actual shape of the part does not always precisely match the expected design. In addition to manual control, there is the option to use a program that communicates with the robot and the welding controller in a read/write mode.

In summary, it is possible to monitor the printing process and also to influence it automatically. Using a laser sensor attached to the robot's end-effector, real-time data about the part is collected within the visible area.

The current challenge is the automated printing of a wall that will very closely correspond in shape to the intended design. At present, welders must intervene in the printing process and independently determine "How should the robot be moved?" In the future, a program will be developed for collecting and processing data, and for executing the necessary actions to answer this question automatically.

Note: only the "data collection and processing" part will be implemented in the course project.

# PROJECT TASK

## Purpose of the work

The purpose of this work is to develop a system for calculating and visualizing the distributed height of layers during 3D printing using the WAAM method. Visual design for diagrams provided here will be realized using **PlantUML**. For application development programming language **Julia** will be used.

- **PlantUML** is a highly versatile tool that facilitates the rapid and straightforward creation of a wide array of diagrams.
- **Julia** is a high-level, general-purpose dynamic programming language, designed to be fast and productive, for e.g. data science, artificial intelligence, machine learning, modeling and simulation, most commonly used for numerical analysis and computational science.

## The task for the coursework

1. Develop a visual model of the system (business model, requirements model, analysis model, design model, component model, deployment model).
2. Select an OO language and programming environment, and create an executable system code.
3. Design the graphical user interface using the graphical library of the programming environment.
4. Define the quality of the visual model and the executable code of the system. Evaluate the model, using the following set of metrics:
    a. Chidamber and Kemerera;
5. Technology development process: a unified incremental process. The number of iterations is at least three.
6. Materials provided:
    a. Report;
    b. Visual model file;
    c. An executable system file

# PROJECT ANALYSIS

## Specification

This system should display graphs with metrics, based on which conclusions can be drawn. For example, that the nozzle should be lowered by 5 mm and the WFS should be increased by 2 times to transfer more metal to a local point with coordinates P(0,0,10), taken from the graph.

## Functional requirements

The Employee must be able to:

- Zoom the selected plot.
- Pan across the selected plot.
- Get the coordinates of a selected point on the plot.
- Rotate the 3D plot around the X, Y, Z axes.
- View 3 types of plots:
    - Point cloud plot
    - Delta height distribution plot
    - Height distribution plot
- Select a folder to monitor for new scans.

## Non-functional requirements

Since the final product is a desktop application, the computer on which the product will run must have:

- Windows 10/11 OS;
- 8 GB of memory;
- 64-bit operating system, x64-based processor;
- 1 GB of free disk space;

### System Users

The main users of this system will be Additive Lab employees who monitor the printing process.

## INCEPTION PHASE

### Business requirements

To describe the functional requirements, a User Story map and Acceptance criteria will be used.

**User Stories: As a … , I want … , so that … .**

- **US1:** As an Employee, I want to see the coordinates of a selected point on the 2D height distribution graph, where:
    - **r** – represents the position along the length of the scanned bead, ranging from 0 to N (where N is the length of that specific bead), measured in millimeters (mm).
    - **h** – represents the dependent variable, indicating the height at position r, measured in millimeters (mm).

  So that I can observe the layer height variation across different bead lengths within the range (0,Nmax), where Nmax is the length of the longest bead being considered.

- **US2:** As an Employee, I want to see the coordinates of the selected point on the 2D graph of the distribution of height changes.
    - **r** – is a value from 0 to N, where N is the length of the scanned bead. Unit of measurement mm.
    - **Δh** – is a variable dependent on r, which displays the height difference between the current height and the previous one at point r. Unit of measurement mm.

  So that I can observe the changes in height changes for different bead lengths in the range (0;N), where N is the length of the longest bead.

- **US3:** As an Employee, I want to see a 3D plot of all the points obtained during printing, so that I can assess the current state of the printed structure.
- **US4:** As an Employee, I want to see the current shape of the printed wall so that I can evaluate the wall's height distribution and determine if the printed part conforms to the wall shape, without needing to look at it in reality.

**Acceptance criterias: Given … , when … , then … .**

- **AC1:** Employee wants to know the coordinates of a point on the plot.
    - **Given:** The Employee is focused on a specific plot.
    - **When:** The Employee hovers the mouse cursor over a point on the plot.
    - **Then:** A tooltip or text box displaying the current coordinates of the point is displayed.
- **AC2:** Employee wants to see how points are distributed in a zoomed-in area of the plot.
    - **Given:** The Employee is focused on a specific plot.
    - **When:** The Employee scrolls the mouse wheel forward/backward.
    - **Then:** The plot zooms in/out accordingly.
- **AC3:** Employee wants to pan across the 2D plot.
    - **Given:** The Employee is focused on a specific 2D plot.
    - **When:** The Employee presses and holds the right mouse button and moves the mouse across the plot.
    - **Then:** The points on the plot shift following the direction of mouse movement.
- **AC4:** Employee wants to pan across the 3D plot.
    - **Given:** The Employee is focused on the 3D plot.
    - **When:** The Employee presses and holds the left mouse button and moves the mouse across the plot.
    - **Then:** The points on the plot shift following the direction of mouse movement.
- **AC5:** Employee wants to rotate the 3D plot to view it from different angles.
    - **Given:** The Employee is focused on the 3D plot.
    - **When:** The Employee presses and holds the right mouse button and moves the mouse across the plot.
    - **Then:** The plot rotates following the direction of mouse movement.
- **AC6:** Employee wants to pan across the 3D plot.
    - **Given:** The Employee is focused on the 3D plot.
    - **When:** The Employee presses and holds the left mouse button and moves the mouse across the plot.
    - **Then:** The points on the plot shift following the direction of mouse movement.

# ELABORATION PHASE

## Use Case Diagram

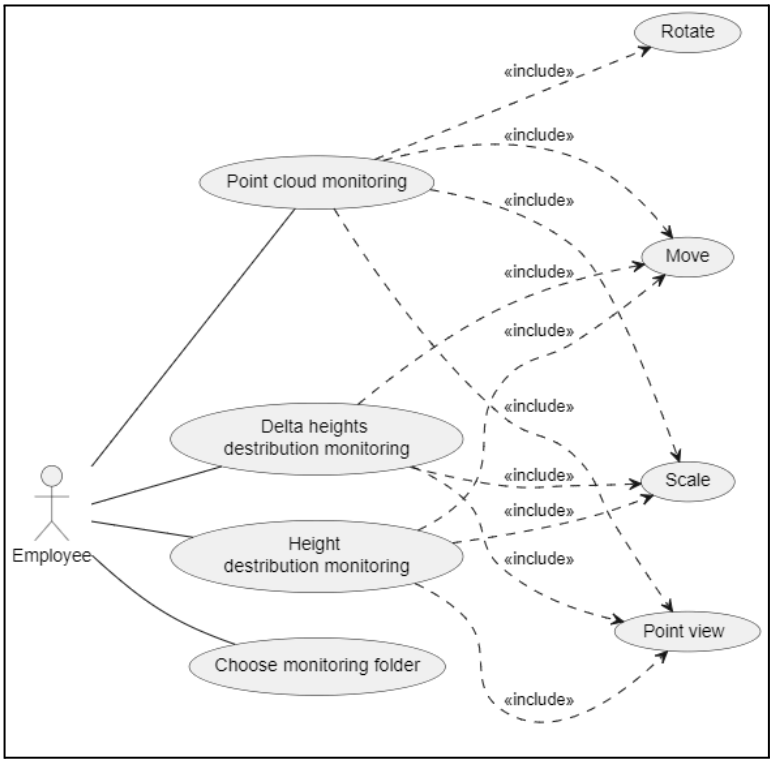



*Figure 1. Use Case Diagram.*

**Actor:**

- Employee - Entity that uses the application.

**Use Case Elements:**

- Point cloud plot monitoring – the user examines the point cloud plot. This feature includes the following capabilities:
    - Rotate: Allows rotating the plot view.
    - Move (Pan): Allows moving the plot view (panning).
    - Scale (Zoom): Allows scaling the plot view (zooming in/out).
    - Point view: Allows displaying information (e.g., coordinates) for selected points.
- Delta height distribution monitoring – the user examines the plot. This feature includes the following capabilities:
    - Move (Pan): Allows moving the plot view (panning).
    - Scale (Zoom): Allows scaling the plot view (zooming in/out).
    - Point view: Allows displaying information (e.g., coordinates) for selected points.

- Height distribution monitoring – the user examines the plot. This feature includes the following capabilities:
    - Move (Pan): Allows moving the plot view (panning).
    - Scale (Zoom): Allows scaling the plot view (zooming in/out).
    - Point view: Allows displaying information (e.g., coordinates) for selected points.
- Choose a monitoring folder – selection of a folder that gets populated with new data each time a scan is completed.

## Scenarios for Use Case Elements

### Point cloud plot monitoring

| | |
|---|---|
| Actor | Employee |
| Goal | The Employee must be able to observe all points obtained from one complete scan, as well as points from previous scans. |
| Preconditions | New data has arrived in the monitoring folder. |
| Successful scenario | 1. Scanned data arrives in the monitoring folder.<br>2. Load files LaserCoordinateN.csv and RobotCoordinateN.csv, where N is the sequential scan number.<br>3. If N = 1: Calculate and save the plane equation(s).<br>4. Display the point cloud on the plot.<br>5. Enable interaction with the plot using the mouse:<br>5.1. Rotate<br>5.2. Move (Pan)<br>5.3. Scale (Zoom)<br>6. Point view (Display point coordinates/info)<br>7. Wait for new data in the monitoring folder. |
| Result | The plot can be interacted with using the mouse, the point cloud is displayed on the plot, and the capability to select a point on the plot is available. |

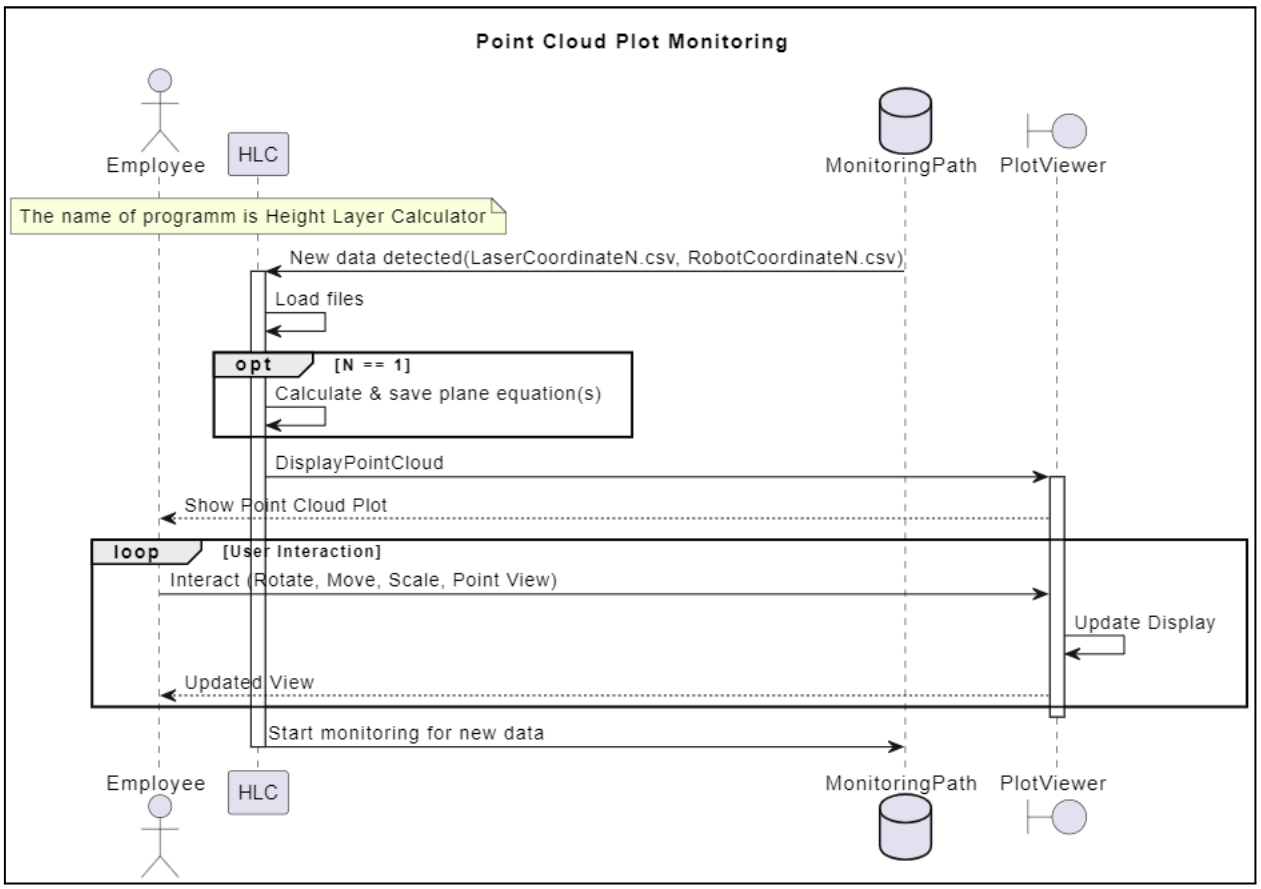



*Figure 2. Point Cloud Sequential Diagram.*

**Current height distribution monitoring**

| | |
|---|---|
| Actor | Employee |
| Goal | The Employee must be able to observe the current and previous height distribution of the layers. |
| Preconditions | New data has arrived in the monitoring folder. |
| Successful scenario | 1. Scanned data arrives in the monitoring folder.<br>2. Load files LaserCoordinateN.csv and RobotCoordinateN.csv, where N is the sequential scan number.<br>3. If N > 1:<br>3.1. Optimize the highest laser sensor points (x[1028] and z[1028]) belonging to the last layer.<br>3.2. Transform the laser sensor points into a point cloud.<br>3.3. Filter out points not belonging to the plane.<br>3.4. If N = 2: |

| | |
|---|---|
| | 3.4.1. Calculate the boundary for grid extrapolation.<br>3.5. Extrapolate z-coordinates onto the grid.<br>3.6. Calculate heights.<br>3.7. Display the resulting points (representing height distribution) on the plot.<br>4. Enable interaction with the plot using the mouse:<br>4.1. Point view (Display coordinates/info)<br>4.2. Move (Pan)<br>4.3. Scale (Zoom)<br>5. Wait for new data in the monitoring folder. |
| Result | The user can interact with the plot using the mouse; the plot displays the r and h axes along with the calculated points, and the ability to select points on the plot is enabled. |

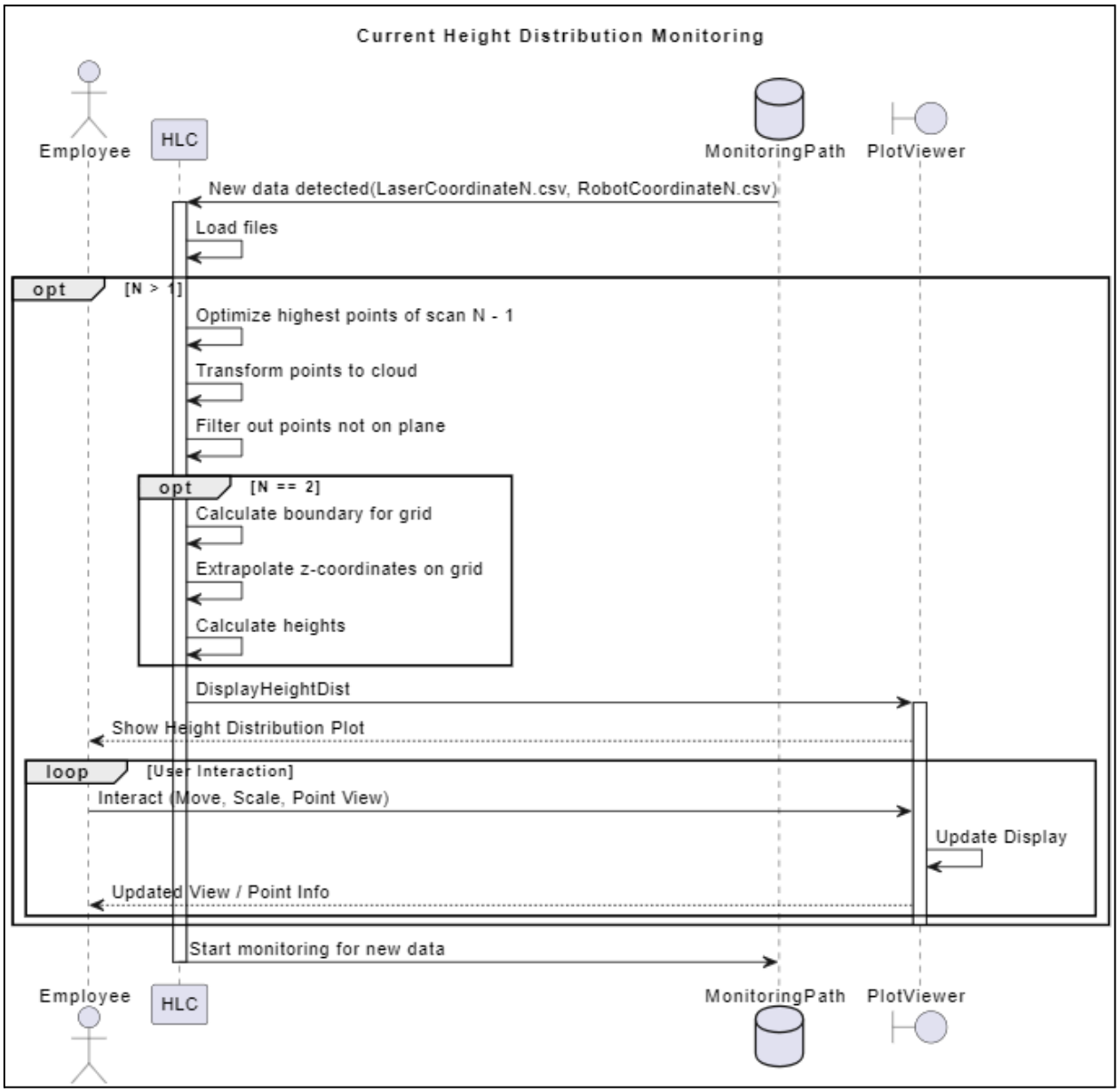



*Figure 3. Current Height Distribution Sequential Diagram.*

**Delta heights distribution monitoring**

| | |
|---|---|
| Actor | Employee |
| Goal | The Employee must be able to observe the current and previous **changes in the height distribution** of the layers (delta height distribution). |
| Preconditions | New data has arrived in the monitoring folder. |
| Successful scenario | 1. Scanned data arrives in the monitoring folder. |

| | |
|---|---|
| | 2. Load files LaserCoordinateN.csv and RobotCoordinateN.csv, where N is the sequential scan number.<br>3. If N > 1:<br>3.1. Optimize the highest laser sensor points (x[1028] and z[1028]) belonging to the last layer.<br>3.2. Transform the laser sensor points into a point cloud.<br>3.3. Filter out points not belonging to the plane.<br>3.4. If N = 2:<br>3.4.1. Calculate the boundary for grid extrapolation.<br>3.5. Extrapolate z-coordinates onto the grid.<br>3.6. Calculate the change in height relative to the previous layer (Δh).<br>3.7. Display the resulting points (representing delta height distribution) on the plot.<br>4. Enable interaction with the plot using the mouse:<br>4.1. Point view (Display coordinates/info)<br>4.2. Move (Pan)<br>4.3. Scale (Zoom)<br>5. Wait for new data in the monitoring folder. |
| Result | The user can interact with the plot using the mouse; the plot displays the r and Δh axes along with the calculated points, and the ability to select points on the plot is enabled. |

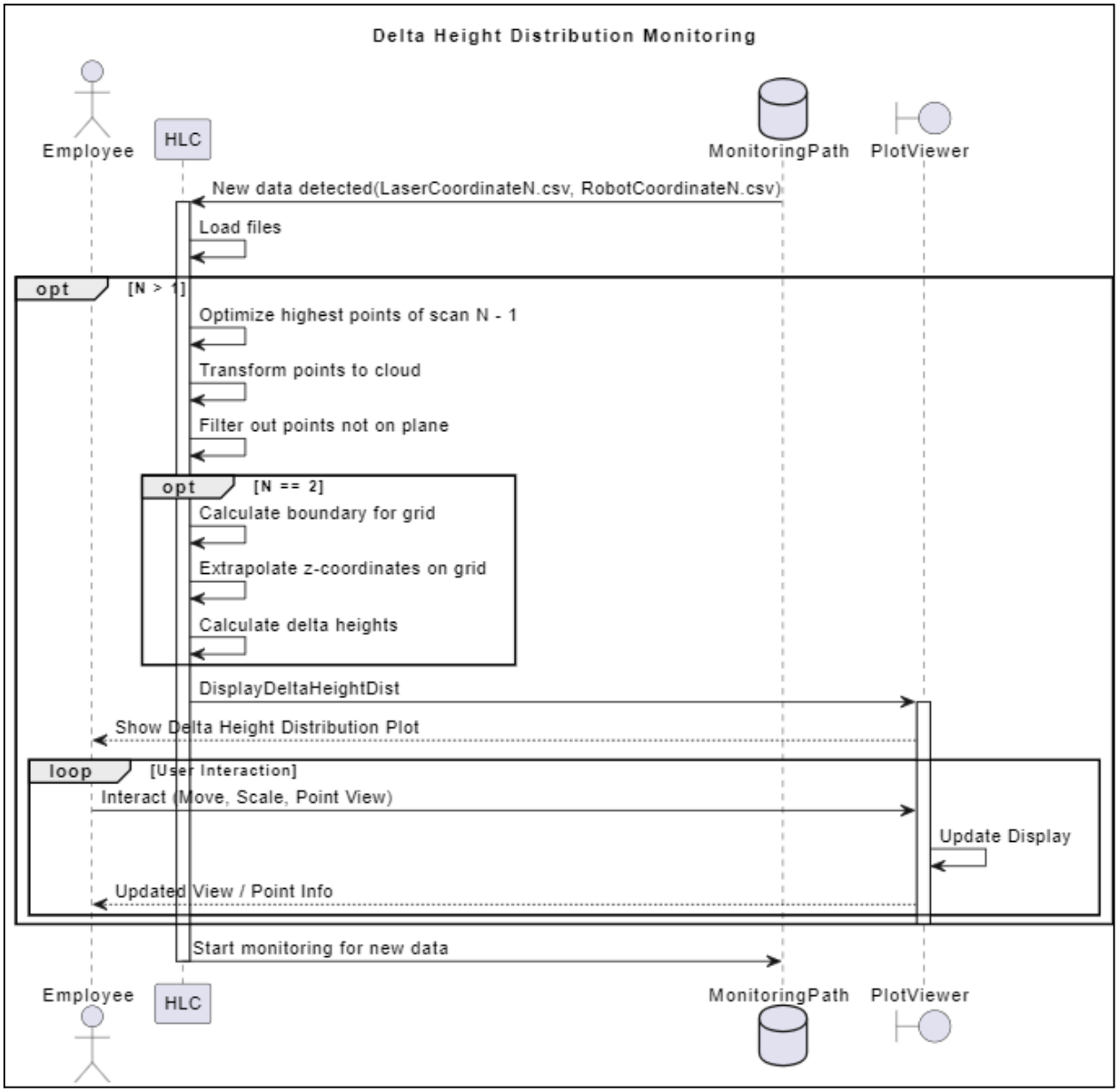



*Figure 4. Delta Height Distribution Sequential Diagram.*

**Choose monitoring path**

| | |
|---|---|
| Actor | Employee |
| Goal | The Employee must be able to select the monitoring folder. |
| Preconditions | The program is running / The application has been launched. |
| Successful scenario | 1. The scanning program (ScannerGUI.exe) is launched.<br>2. A new folder with the format YYYY-MM-dd_HH-mm-ss is created (example 2025-03-20_14-52-45). |

| | |
|---|---|
| | 3. The main program starts and prompts the user to select a folder.<br>4. The user selects a folder.<br>5. If the selected folder is valid (matches the expected format/exists):<br>5.1. Check for new scan data: If the folder contains new RobotCoordinateN.csv and LaserCoordinateN.csv files where N > 1:<br>5.1.1. Process the previous scan's data: Calculate points and visualize them from files RobotCoordinateM.csv and LaserCoordinateM.csv, where M = N - 1.<br>5.1.2. Wait for t seconds (example t = 2 seconds)<br>5.1.3. Repeat step 5.1.<br>6. Program termination. |
| Result | The program displays the path of the currently selected monitoring folder. |

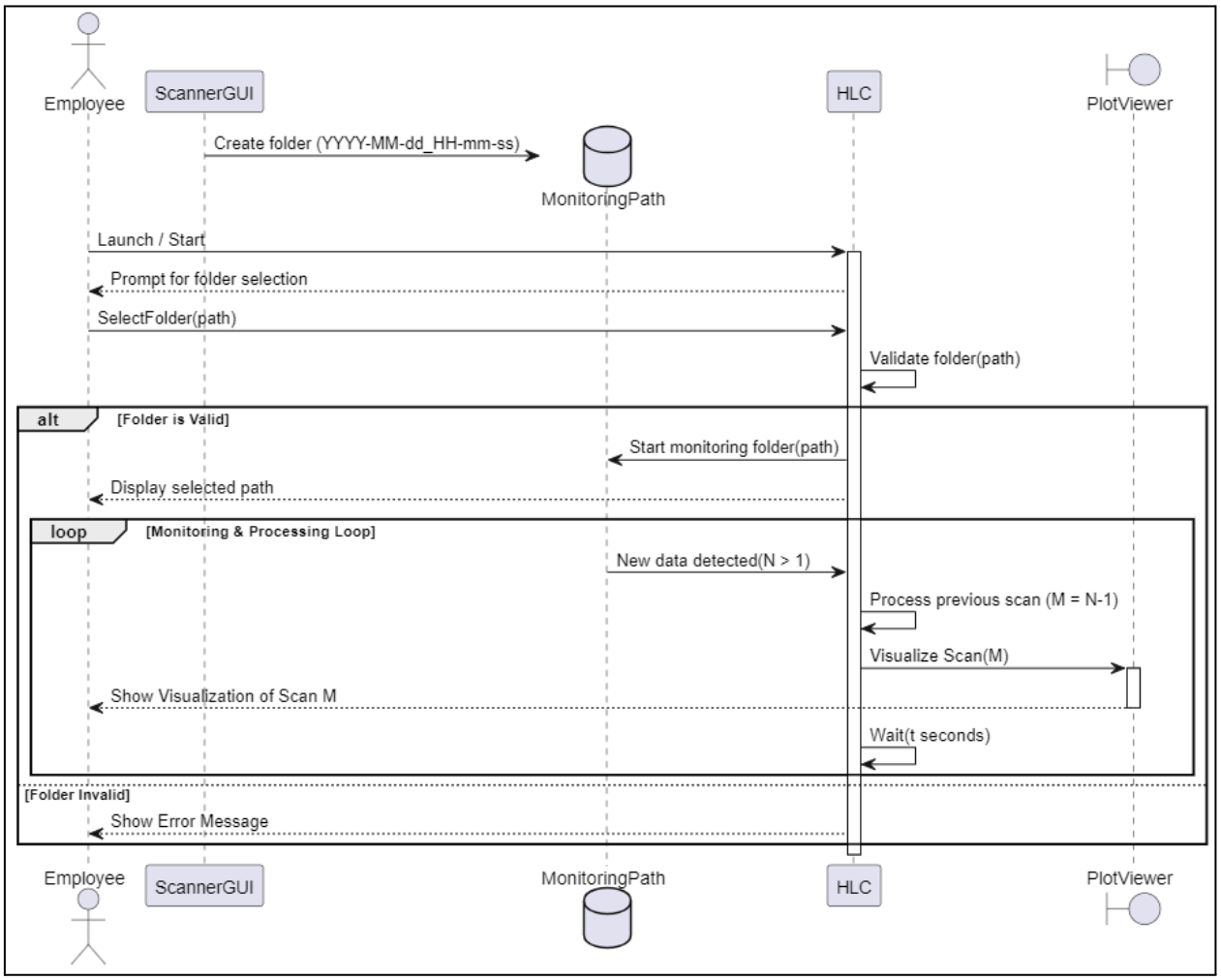



*Figure 5. Monitoring Path Sequential Diagram.*

## Class Diagram

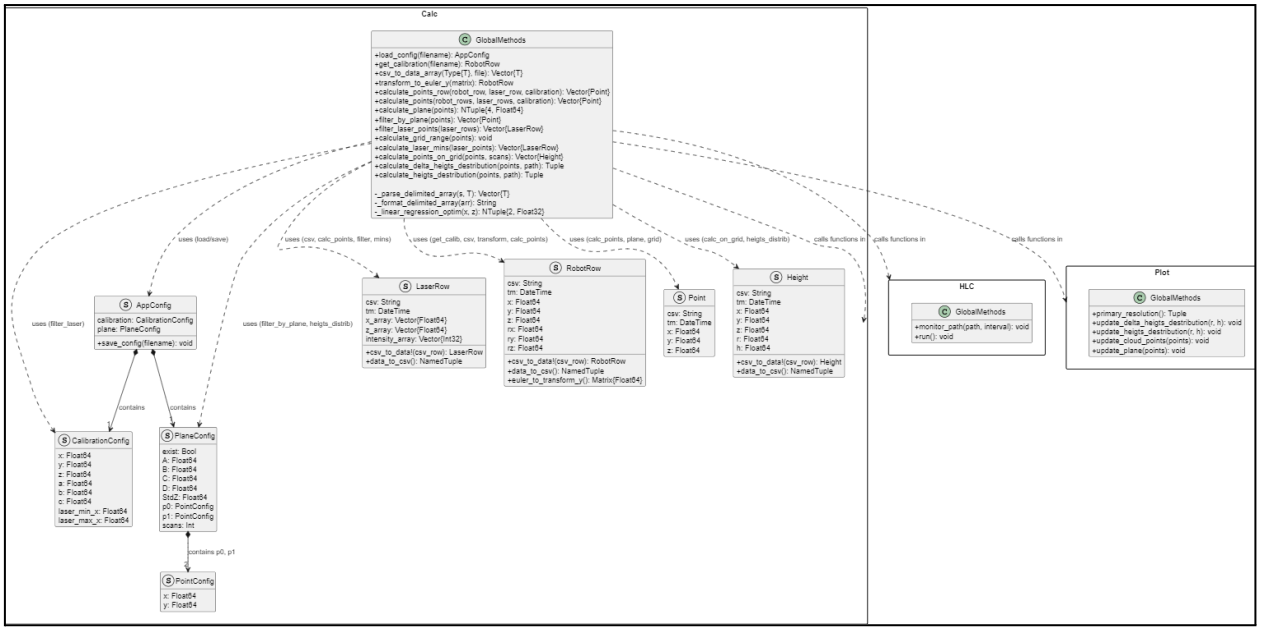



*Figure 6. Class Diagram.*

### Description of classes

Classes (Structs):

- PointConfig: Structure for storing 2D coordinates (X, Y) used internally by PlaneConfig.
- CalibrationConfig: Stores system calibration parameters including sensor offset, angles, and laser range.
- PlaneConfig: Contains parameters for the calculated reference plane including equation coefficients, points for defining the range, and the number of grid steps.
- AppConfig: Root configuration structure that combines CalibrationConfig and PlaneConfig.
- LaserRow: Represents a single data set (scan) from a laser scanner, including a timestamp and arrays of X, Z coordinates, and point intensities.
- RobotRow: Represents the position and orientation (pose) of a robot at a given time, or calibration parameters.
- Point: Represents a single point in the 3D world coordinate system with a timestamp and source ID.
- Height: Represents a point on the calculated height grid, including grid coordinates (X, Y), interpolated Z, coordinate along the grid (R), and height (H) above the base plane or previous layer.

- GlobalMethods: An artificial class (not a real struct in the code) used in this diagram to group all the design functions that are not methods of specific structs. Contains the core application logic, read/write, calculation, and graph updating functions.

## Quality estimation

Collecting a full set of quality metrics (WMC, NOC, CBO, RFC, LCOM, DIT, NC, NOM, LOC) for Julia code automatically can be a non-trivial task, since the static analysis tool ecosystem for Julia is still evolving compared to languages like Java or C#. To collect the LOC metric, the code metrics_collector.jl was written, it output:

```
Analyzing directory: C:\Users\Admin\Projects\Additive-Lab-Software\CourceProject3\HLC\src

--- Basic Code Metrics ---
Total files processed: 8
-------------------------
Total Lines (TLOC):      1955
Blank Lines:             315
Comment Lines (approx.): 319
Code Lines (approx.):    1321
-------------------------
Struct Count (NC approx.):    8
Function Count (NOM approx.): 34
-------------------------
NOC (Number of Children):  0 (assuming no struct inheritance)
DIT (Depth of Inheritance): 0 (assuming no struct inheritance)
-------------------------
NOTE: WMC, CBO, RFC, LCOM metrics require more advanced
      static analysis tools (e.g., SciTools Understand)
      and cannot be reliably calculated by this script.
-------------------------
```

*Figure 7. Basic Metrics.*

## Quality Estimation

| Metric | WMC (Est.) | NOC | CBO (Est.) | RFC (Est.) | LCOM (Est.) |
|---|---|---|---|---|---|
| PointConfig | 0 | 0 | 1 | 0 | 0 |
| CalibrationConfig | 0 | 0 | 2 | 0 | 0 |
| PlaneConfig | 0 | 0 | 3 | 0 | 0 |
| AppConfig | 1 | 0 | 3 | 1 | 0 |
| LaserRow | 2 | 0 | 1 | 2 | 0 |
| RobotRow | 3 | 0 | 1 | 3 | 0 |
| Point | 0 | 0 | 1 | 0 | 0 |
| Height | 2 | 0 | 1 | 2 | 0 |
| GlobalMethods | 26 | 0 | >=11 | >=41 | ~0.1 |
| Average value | ~3.8 | 0 | ~2.8 | ~5.4 | ~0.01 |

## System metrics

| Metric | Value | Notes |
|---|---|---|
| DIT (Depth of Inheritance Tree) | 0 | From script/analysis |
| NC (Number of Classes) | 8 | From script, excluding GlobalMethods |
| NOM (Number of Methods) | 34 | From script |
| LOC∑ (Lines of Code) | 1321 | Approximate value of lines of code from script |

## Component diagram

Component diagram visualize the organization of system components and the dependencies between them. It allows to get a high-level view of the components of the system.

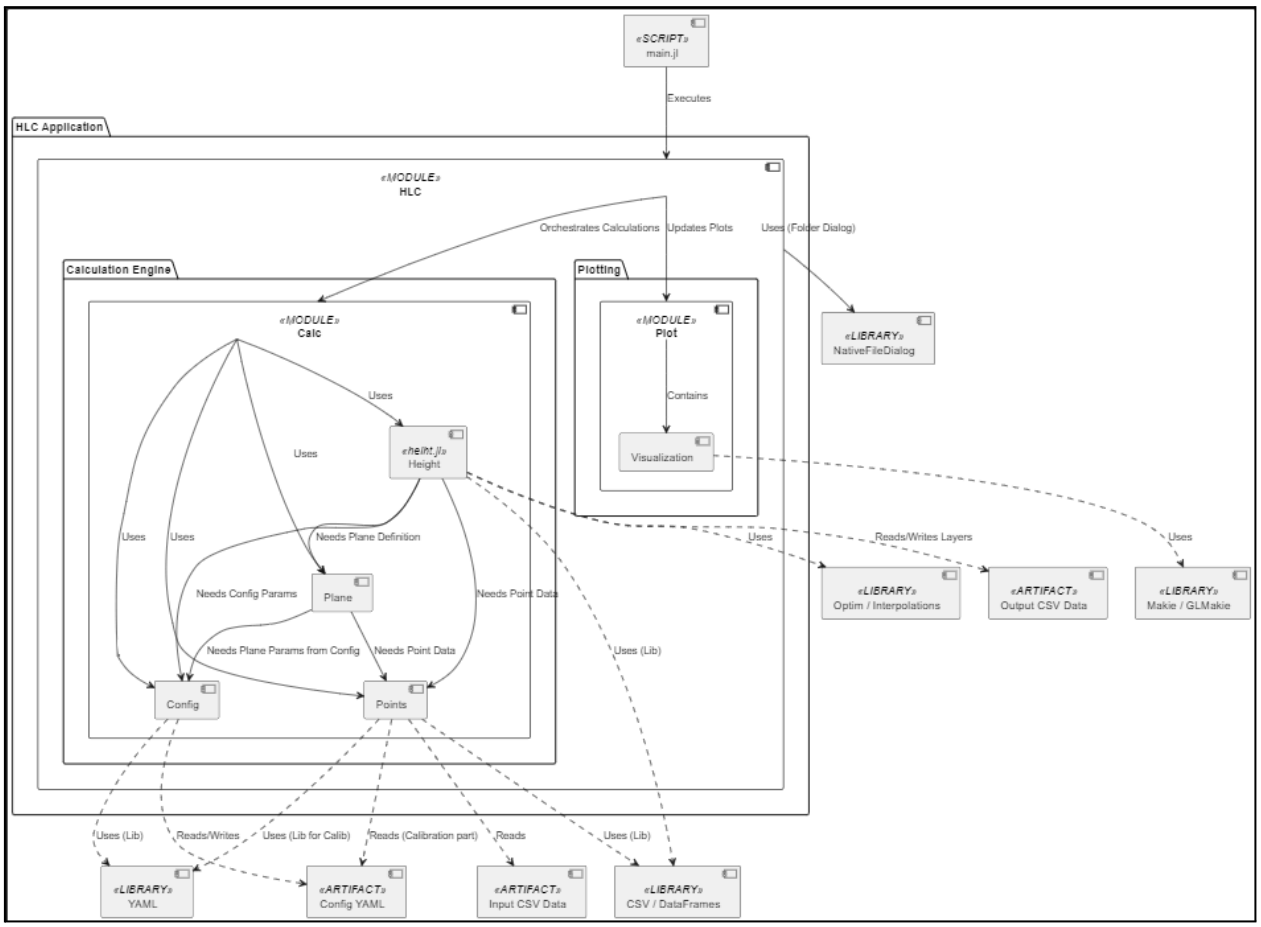



*Figure 8. Component Diagram*.

## CONSTRUCTION PHASE

The construction phase of the unified development process is, as the name implies, responsible for transforming the architectural model established in the previous stages into working source code that will make use cases operational for end users (Pressman, 2009, p. 56). Each iteration is in-tended to yield a working and tested version of the application.

### The 1st Iteration

In the first iteration, the Plot module was implemented. This module is responsible for visualizing points on different graphs, which allows for visual testing of the logic of other modules during their implementation by directly calling the graph plotting methods. This approach is viable because creating tests for very large datasets can be highly time-consuming. Such a testing strategy is feasible because there is a clear understanding of the collected data (a test dataset was successfully visualized using a specialized program, and this visualization is considered the target output for the Plot module).

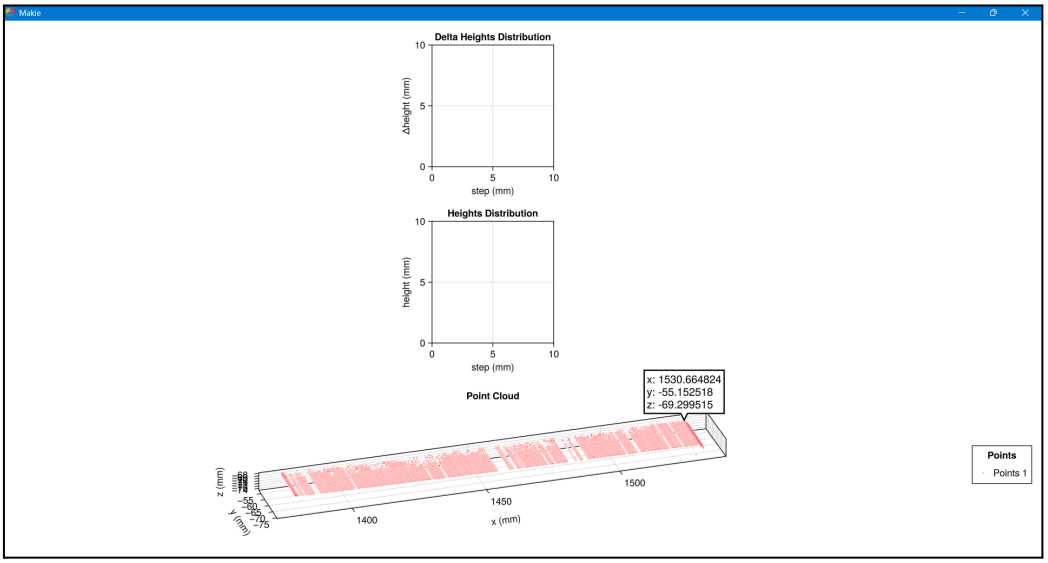



*Figure 9. Plane simulation.*

An interesting aspect during the implementation of this module was the section of code responsible for creating the legend. Julia is considered a relatively new programming language, so it was necessary to use "workarounds" (colloquially, "crutches") to achieve this functionality.

```
function update_delta_heigts_destribution(r, h)

    . . .

    if !isnothing(HLC_AX_HD_LEGEND[]) && HLC_AX_HD_LEGEND[].parent !== nothing
```

```
        try
            delete!(HLC_AX_HD_LEGEND[]) # Delete legend
            HLC_AX_HD_LEGEND[] = nothing
        catch e
            @warn "Could not delete previous legend object: $e"
            HLC_AX_HD_LEGEND[] = nothing
        end
    end

    . . .

    new_legend = Legend(HLC_FIG, HLC_AX_HD, "ΔLayers")
    HLC_FIG[1, 2] = new_legend
    HLC_AX_HD_LEGEND[] = new_legend

    # Display the Figure
    display(HLC_FIG)
end
```

Despite this particular challenge, it is worth highlighting a specific line of code that adds the capability to inspect each data point on every graph. Notably, only a single line of code is used to accomplish this task.

```
HLC_INSPECTOR = DataInspector(HLC_FIG)
```

### The 2st Iteration

In the second iteration, the Calc module was implemented. This module is responsible for processing and calculating the graph data that was described earlier. The second iteration was divided into four stages:

- Creation of a parser for the configuration file.
- Development of algorithms for point filtering.
- Creation of a parser for data structures.
- Development of algorithms for calculating:
    - Wall height
    - Layer height
    - Point cloud (this requires transformation coefficients)

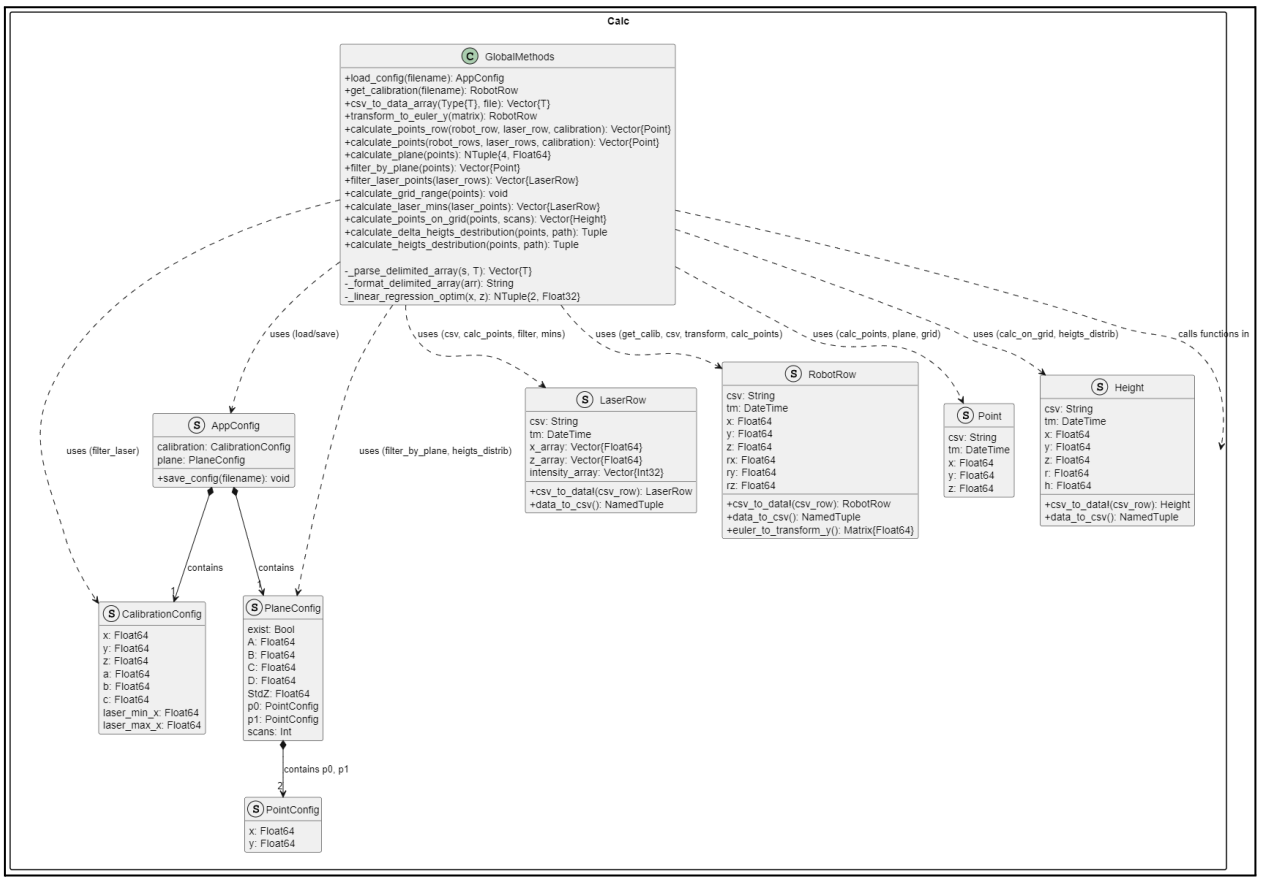



*Figure 10. Calc Component Diagram.*

The most important and interesting stage was the extrapolation of points onto an (X, Y) grid. All subsequent calculations, and ultimately the data analysis itself, are based on this stage. Therefore, during the algorithm development process, it was necessary to study existing libraries for point interpolation that could meet the performance and reliability requirements. Through trial and error, the initial algorithm was modified, and the Interpolations library was used for its implementation.

```
function calculate_points_on_grid(points::Vector{Point},
scans::Int64)::Vector{Height}

    . . .

    itp = Interpolations.linear_interpolation(xy_sorted, z_sorted)

    . . .


    for i in 1:length(x_grid)

        . . .
```

```
            height.z = itp(xy_grid[i])

      . . .

        push!(heights, height)
    end
    return heights
end
```

### The 3st Iteration

The third stage encompasses the integration of the two previous ones. To utilize these two independent modules, an HLC module is employed. This module provides the interface between the user and the program. Upon program launch, at the main entry point, the user is given the option to select a working directory from which new data (real/test data) will be collected and processed.

A key method is named monithor_path, which utilizes the Builder design pattern – it combines independent components to construct huge object. Calling the run() method initiates the HLC program.

```
function monitor_path(path::String, interval::Int64)

         . . .

                # Report readiness of the PREVIOUS index (if not zero)
                if last_confirmed_index > 0
                    prev_index = last_confirmed_index
                    robot_file_prev = joinpath(monitored_dir,
"RobotCoordinate$(prev_index).csv")
                    laser_file_prev = joinpath(monitored_dir,
"LaserCoordinate$(prev_index).csv")

                    # read laser and robot points
                    robot_rows = Calc.csv_to_data_array(Calc.RobotRow,
robot_file_prev)
                    laser_rows = Calc.csv_to_data_array(Calc.LaserRow,
laser_file_prev)
                    laser_rows = Calc.filter_laser_points(laser_rows)

                    # calculate points
                    cloud_points = Calc.calculate_points(robot_rows,
laser_rows, calibration)

                    # firts scan - plane
```

```
                    if prev_index == 1
                        # calculate plain and save to config.yaml
                        plane_params = Calc.calculate_plane(cloud_points)

                        # update plots (with plane)
                        Plot.update_plane(cloud_points)
                    else
                        # find max approximated
                        laser_rows_mins = Calc.calculate_laser_mins(laser_rows)

                        # calculate points and filter it by plane
                        cloud_points_mins = Calc.calculate_points(robot_rows,
laser_rows_mins, calibration)
                        cloud_points_mins =
Calc.filter_by_plane(cloud_points_mins)

                        # interpolate it, create grid
                        if prev_index == 2
                            Calc.calculate_grid_range(cloud_points_mins)
                        end
                        heights_on_grid::Vector{Calc.Height} =
Calc.calculate_points_on_grid(cloud_points_mins, conf.plane.scans)

                        # create delta heights description and current heights
description
                        x_delta, y_delta, z_delta, r_delta, h_delta =
Calc.calculate_delta_heigts_destribution(heights_on_grid, monitored_dir)
                        x, y, z, r, h =
Calc.calculate_heigts_destribution(heights_on_grid, monitored_dir)

                        # plot it
                        Plot.update_delta_heigts_destribution(r_delta, h_delta)
                        Plot.update_heigts_destribution(r, h)
                        Plot.update_cloud_points(cloud_points)
                    end

                elseif next_index_to_check == 1 # Special case for the first
found index
                    @info "This is the first detected pair (index 1). No
previous files."
                end

                # Update state
                last_confirmed_index = next_index_to_check

                # DO NOT wait for the interval, immediately check the next
index,
                # as multiple pairs might have appeared simultaneously
                @info "Scan with index: $next_index_to_check was processed!"
                continue
```

```
            else
                # Pair for next_index_to_check is not ready yet (or doesn't
exist)
                @debug "Pair for index $next_index_to_check not found.
Waiting..."
                # Wait before the next check
                sleep(interval)
            end

        . . .

end
```

## Summary

Ultimately, all business requirements were implemented and successfully tested in real-time. During the use of this program, new requirements have emerged which will be implemented in a future version of the program. Additionally, due to the newness of the Julia programming language, the program has some bugs and overheads that should ideally be fixed before new features are added.

# CONCLUSIONS

The main objective of this project was to develop a Julia-based desktop application for Additive Lab, allowing employees to analyze metal 3D printing data using interactive graphs and metrics to optimize the printing process.

Built using Julia, the system includes Plot, Calc (using the Interpolations library for tasks such as point extrapolation) and HLC modules for visualization, data manipulation and user interaction on Windows platforms.

The result is a functional desktop application that meets the main requirements. It allows users to visualize various print data graphs such as point clouds and height distributions, with interactive capabilities such as zooming, panning, 3D rotation and checking point coordinates, as well as monitoring selected folders for new scan data. All critical features were successfully implemented and tested in real time.

To summarize, the project successfully achieved its main objectives and OOP was also used.

## REFERENCES


1. **Visual Studio Code** [https://code.visualstudio.com/]: Primary Integrated Development Environment (IDE) for developing and managing the project's Julia code.
2. **PlantUML** [https://plantuml.com/]: Tool for creating UML diagrams (e.g., for system design and architecture) to aid in application design and documentation.
3. **Official Julia Programming Language Website** [https://julialang.org/]: Main resource for official Julia language documentation, packages, and information.
4. **Interpolations.jl Documentation** [https://juliamath.github.io/Interpolations.jl/stable/]: Documentation for the Interpolations.jl library, used in the Calc module for data point interpolation and extrapolation.
5. **Google Gemini** [https://gemini.google.com/]: Utilized for text translation.
6. **HLC Project GitHub Repository** [https://github.com/IGORSVOLOHOVS/HLC.git]: The project's source code repository (housing the HLC module and/or full application) for version control and storage.
7. **AdditiveLab** [https://tsi.lv/research/research-at-tsi/infrastructure/additivelab-additive-manufacturing-research-and-solutions-laboratory/]: Information on the Additive Lab at TSI, the target user environment, which helped in understanding user context and project requirements.
8. **Makie.jl Documentation** [https://docs.makie.org/stable/]: Documentation for the Makie.jl plotting library, likely used or considered for the Plot module for creating 2D and 3D visualizations.

# ANNEXE

## Screenshots

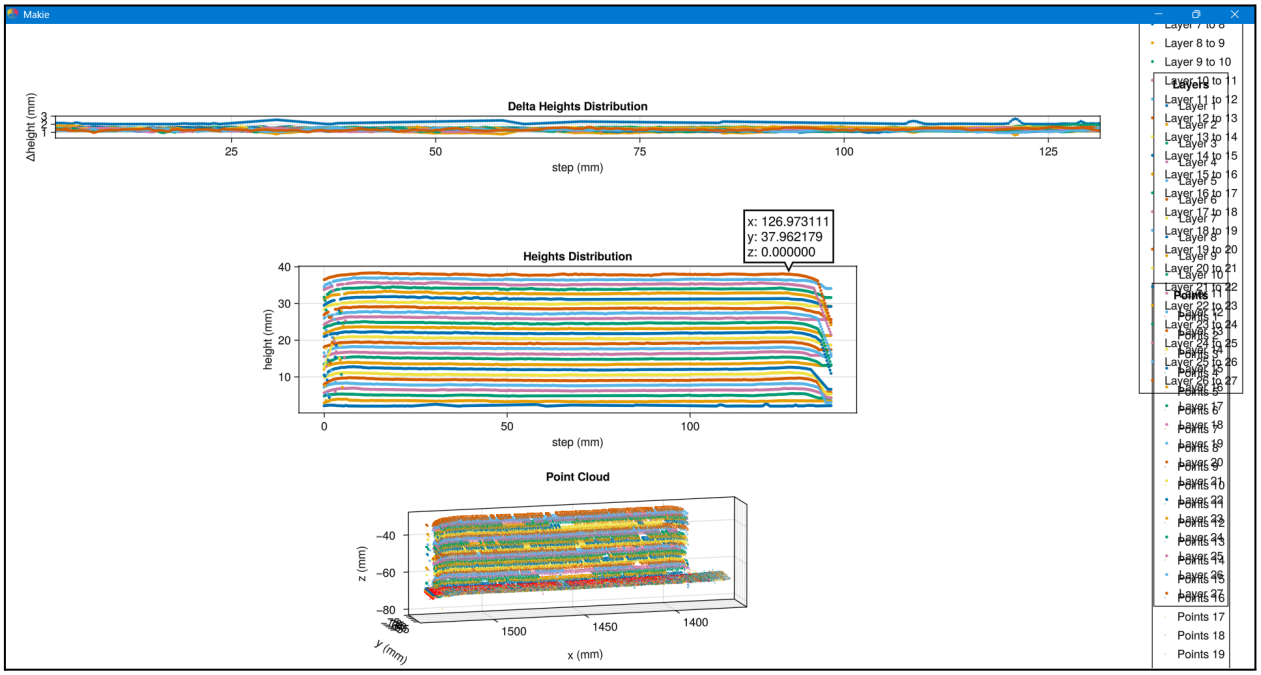



*Figure 12. Real testing (good wall with 27 layers).*

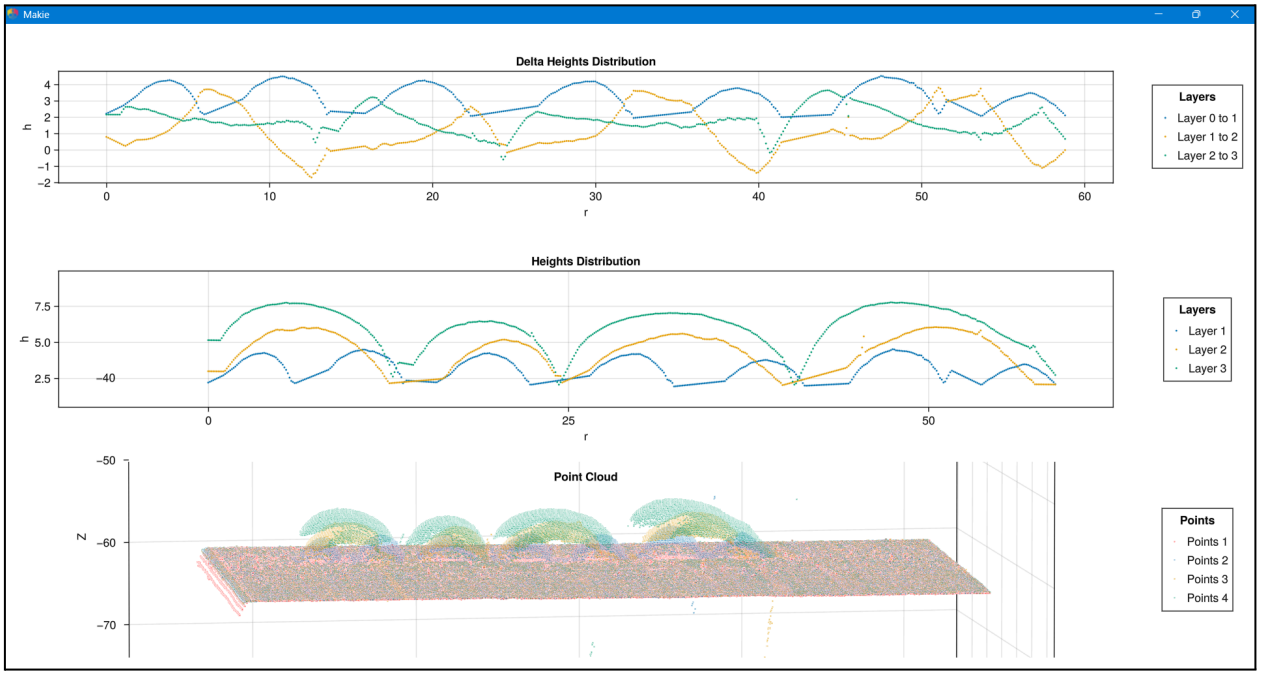



*Figure 13.Test data (bad wall with 3 layers).*

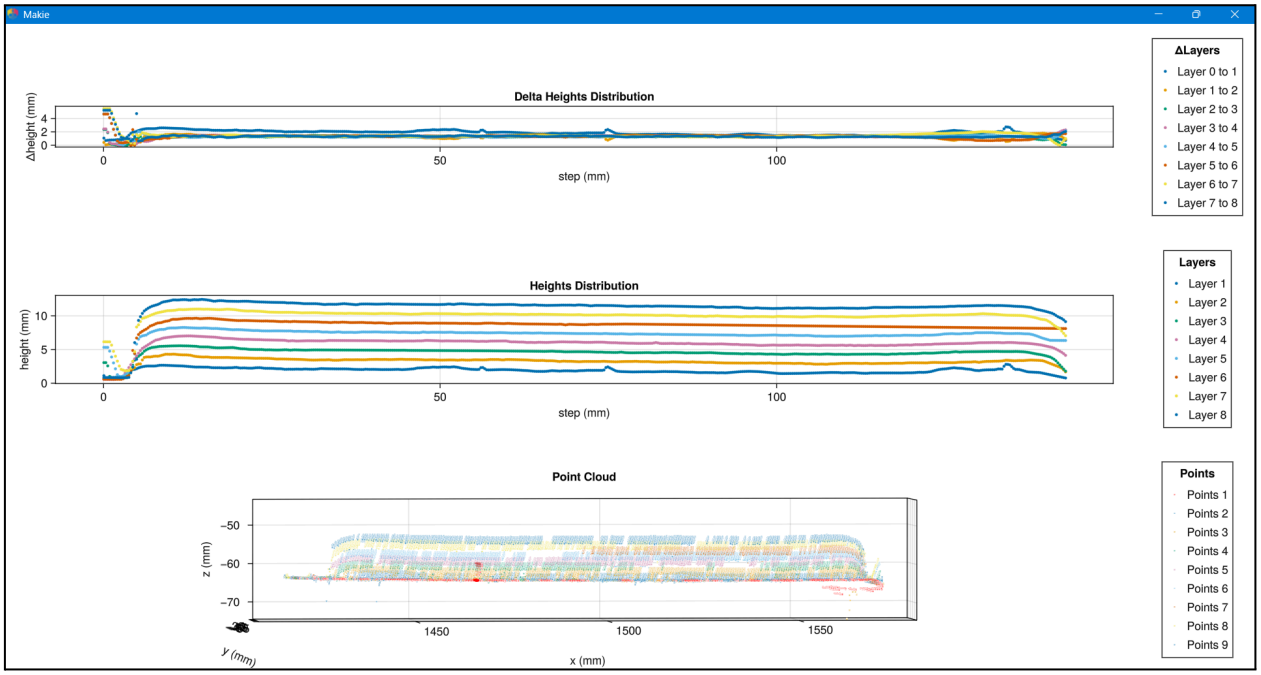



*Figure 13.Real-time data (See Figure 14).*

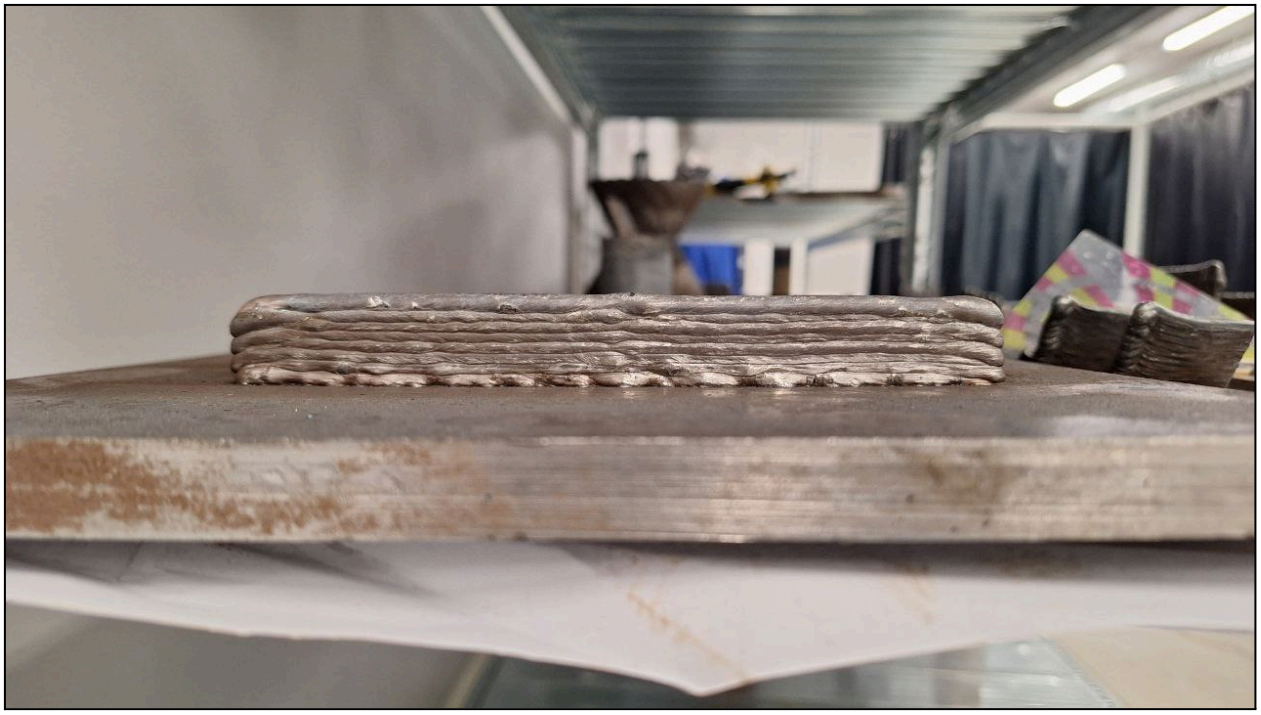


*Figure 14. Metal wall with 8 layers.*

## Program code

### calc/calc.jl

```
# calc/calc.jl
module Calc
    include("config.jl")
```

```
    include("points.jl")
    include("plane.jl")
    include("heiht.jl")
end
```

**calc/config.jl**

```
# calc/config.jl
using YAML
using Printf # For formatted output
using OrderedCollections # <-- Add this line to import

mutable struct PointConfig
    x::Float64
    y::Float64
end

mutable struct CalibrationConfig
    x::Float64
    y::Float64
    z::Float64
    a::Float64
    b::Float64
    c::Float64
    laser_min_x::Float64
    laser_max_x::Float64
end

mutable struct PlaneConfig
    exist::Bool
    A::Float64
    B::Float64
    C::Float64
    D::Float64
    StdZ::Float64
    p0::PointConfig # Using nested structure
    p1::PointConfig # Using nested structure
    scans::Int
end

mutable struct AppConfig
    calibration::CalibrationConfig
    plane::PlaneConfig
end

# --- Function to load configuration from file ---
"""
    load_config(filename::String="config.yaml") -> Union{AppConfig, Nothing}
```

```
Loads configuration from a YAML file and returns an AppConfig object.
Returns `nothing` in case of an error.
"""
function load_config(filename::String="config.yaml")::Union{AppConfig, Nothing}
    try
        if !isfile(filename)
            @error "Configuration file not found: $filename"
            return nothing
        end

        raw_config = YAML.load_file(filename)

        # Key existence check (basic)
        if !haskey(raw_config, "calibration") || !haskey(raw_config, "plane")
             @error "Missing required sections 'calibration' or 'plane' in file
$filename"
             return nothing
        end

        cal_data = raw_config["calibration"]
        plane_data = raw_config["plane"]

        # Nested key check (example)
        req_cal_keys = ["exist", "x", "y", "z", "a", "b", "c", "laser_min_x",
"laser_max_x"]
        req_plane_keys = ["A", "B", "C", "D", "StdZ", "p0", "p1", "scans"]
        req_point_keys = ["x", "y"]

        # (Optional) Add stricter key validation if necessary

        # Create nested structures
        p0 = PointConfig(plane_data["p0"]["x"], plane_data["p0"]["y"])
        p1 = PointConfig(plane_data["p1"]["x"], plane_data["p1"]["y"])

        # Create main structures
        calibration_config = CalibrationConfig(
            cal_data["x"], cal_data["y"], cal_data["z"],
            cal_data["a"], cal_data["b"], cal_data["c"],
            cal_data["laser_min_x"], cal_data["laser_max_x"]
        )

        plane_config = PlaneConfig(
            plane_data["exist"], # Take 'exist' from the plane section
            plane_data["A"], plane_data["B"], plane_data["C"], plane_data["D"],
            plane_data["StdZ"],
            p0, p1,
            plane_data["scans"]
        )
```

```
        # Assemble the final configuration object
        app_config = AppConfig(calibration_config, plane_config)

        return app_config

    catch e
        @error "Error loading or parsing file '$filename':" exception=(e,
catch_backtrace())
        return nothing # Return nothing on any error
    end
end

"""
    save_config(config::AppConfig, filename::String="config.yaml")

Saves the AppConfig object to a YAML file, preserving key order.
"""
function save_config(config::AppConfig, filename::String="config.yaml")
    # Use OrderedDict to preserve key order at all levels
    config_odict = OrderedDict(
        "calibration" => OrderedDict( # <-- OrderedDict here
            "x" => config.calibration.x,
            "y" => config.calibration.y,
            "z" => config.calibration.z,
            "a" => config.calibration.a,
            "b" => config.calibration.b,
            "c" => config.calibration.c,
            "laser_min_x" => config.calibration.laser_min_x,
            "laser_max_x" => config.calibration.laser_max_x
        ),
        "plane" => OrderedDict( # <-- OrderedDict here
            "exist" => config.plane.exist,
            "A" => config.plane.A,
            "B" => config.plane.B,
            "C" => config.plane.C,
            "D" => config.plane.D,
            "StdZ" => config.plane.StdZ,
            "p0" => OrderedDict( # <-- OrderedDict here too, if x, y order
matters
                "x" => config.plane.p0.x,
                "y" => config.plane.p0.y
            ),
            "p1" => OrderedDict( # <-- OrderedDict here too
                "x" => config.plane.p1.x,
                "y" => config.plane.p1.y
            ),
            "scans" => config.plane.scans # <-- If scans is an array/vector,
order is preserved automatically
        )
    )
```

```
    try
        # YAML.write_file should handle OrderedDict correctly
        YAML.write_file(filename, config_odict)
    catch e
        @error "Error saving configuration to file '$filename':" exception=(e,
catch_backtrace())
    end
end
```

**calc/height.jl**

```
# calc/heiht.jl
using Dates
using Statistics
using CSV
using DataFrames
using LinearAlgebra # For norm (distance between points)
using Optim
using Interpolations
using ScatteredInterpolation

"""
    Height

Structure to store a point with its calculated height relative to a plane.

# Fields
- `csv::String`: Identifier of the data source.
- `tm::DateTime`: Timestamp (may be irrelevant for pure height data).
- `x::Float64`: X coordinate of the grid point.
- `y::Float64`: Y coordinate of the grid point.
- `z::Float64`: Interpolated/nearest Z value of the source points within the
grid cell.
- `r::Float64`: Radius or other parameter associated with the point (e.g., from
the CSV file).
- `h::Float64`: Height of the point above/below the calculated plane (z -
z_plane).
"""
mutable struct Height
    csv::String
    tm::DateTime
    x::Float64
    y::Float64
    z::Float64 # Interpolated/nearest Z
    r::Float64 # Added field r
    h::Float64 # Height above the plane
```

```
    # Corrected default constructor
    Height() = new("", Dates.unix2datetime(0), NaN, NaN, NaN, NaN, NaN) # Added
NaN for r
    # Constructor with values
    Height(csv, tm, x, y, z, r, h) = new(string(csv), tm, Float64(x),
Float64(y), Float64(z), Float64(r), Float64(h)) # Added r
end


# --- Functions for working with CSV ---

"""
    csv_to_data!(row::Height, csv_row::DataFrameRow)

Fills the `Height` structure from a DataFrame row.
Assumes the presence of columns :TimeReceived, :X, :Y, :Z, :R, :H, :CSV.
"""
function csv_to_data!(row::Height, csv_row::DataFrameRow)::Height
    try
        # Time processing (use global formats if Calculation is not available)
        # REPLACEMENT: Assume formats are defined globally or passed as
arguments
        # If they are still in Main.Calculation, leave as is.
        # For example, use strings directly (less efficient):
        local fmt_t = DateFormat("yyyy-mm-ddTHH:MM:SS.s")
        local fmt_space = DateFormat("yyyy-mm-dd HH:MM:SS.s")

        time_val = csv_row.TimeReceived
        if isa(time_val, DateTime)
            row.tm = time_val
        elseif isa(time_val, AbstractString)
            time_str = string(time_val)
            try
                row.tm = DateTime(time_str, fmt_t)
            catch
                try
                    row.tm = DateTime(time_str, fmt_space)
                catch e
                    @warn "Failed to parse TimeReceived '$time_str': $e"
                    row.tm = Dates.unix2datetime(0)
                end
            end
        else
            @warn "Unexpected type for TimeReceived: $(typeof(time_val))"
            row.tm = Dates.unix2datetime(0)
        end

        # Processing numeric fields
        row.x = parse(Float64, string(coalesce(csv_row.X, "NaN")))
        row.y = parse(Float64, string(coalesce(csv_row.Y, "NaN")))
```

```
        row.z = parse(Float64, string(coalesce(csv_row.Z, "NaN")))
        row.r = parse(Float64, string(coalesce(csv_row.R, "NaN"))) # Added
handling for R
        row.h = parse(Float64, string(coalesce(csv_row.H, "NaN")))
        # Processing CSV field if it exists
        row.csv = :CSV in names(csv_row) ? coalesce(csv_row.CSV, "") : ""


    catch e
        @error "Error processing Height from CSV: $e. Row data: $csv_row"
        showerror(stderr, e, catch_backtrace())
        println(stderr)
        # Reset to default values on error
        row.tm = Dates.unix2datetime(0)
        row.x = row.y = row.z = row.r = row.h = NaN # Added r = NaN
        row.csv = ""
    end
    return row
end

"""
    data_to_csv(row::Height)::NamedTuple

Converts the `Height` structure to a NamedTuple for writing to CSV.
Includes fields CSV, TimeReceived, X, Y, Z, R, H.
"""
function data_to_csv(row::Height)::NamedTuple
    # REPLACEMENT: Use format directly or pass as argument
    local fmt_t = DateFormat("yyyy-mm-ddTHH:MM:SS.s")
    return (
        CSV=row.csv,
        TimeReceived=Dates.format(row.tm, fmt_t),
        X=row.x,
        Y=row.y,
        Z=row.z,
        R=row.r, # Added field R
        H=row.h
    )
end

"""
    csv_to_data_array(::Type{Height}, file::String)::Vector{Height}

Reads a CSV file and converts rows into a vector of `Height` structures.
"""
function csv_to_data_array(::Type{Height}, file::String)::Vector{Height}
    result = Height[]
    try
        # REPLACEMENT: Use format directly or pass as argument
        local fmt_t = DateFormat("yyyy-mm-ddTHH:MM:SS.s")
```

```
        # Specify the main date format, csv_to_data! will handle other variants
        df = CSV.read(file, DataFrame; dateformat=fmt_t)
        for csv_row in eachrow(df)
            row_instance = Height()
            push!(result, csv_to_data!(row_instance, csv_row)) # csv_to_data!
now handles R
        end
    catch e
        @error "Failed to read or process Height CSV '$file': $e"
        showerror(stderr, e, catch_backtrace())
        println(stderr)
        return Height[] # Return an empty vector on error
    end
    return result
end


# --- Calculation Functions ---

"""
    calculate_grid_range(points::Vector{Point}) -> NamedTuple{(:xmin, :xmax,
:ymin, :ymax), NTuple{4, Float64}}

Determines the minimum and maximum X and Y coordinates from a vector of `Point`
points.
(Kept from the previous version, may require adaptation depending on usage)
"""
function calculate_grid_range(points::Vector{Point})
    config = load_config()

    # Extracting plane parameters
    config.plane.p0.x = points[1].x
    config.plane.p0.y = points[1].y
    config.plane.p1.x = points[end].x
    config.plane.p1.y = points[end].y

    save_config(config)
end

function _linear_regression_optim(x, z)
    # Define the objective function (SSE)
    function sse(params)
        a, b = params
        predictions = a .* x .+ b
        return sum((z .- predictions).^2)
    end

    # Initial guess for parameters [a, b]
    # Use Float64 for internal calculations
    x_f64 = Float64.(x)
```

```
    z_f64 = Float64.(z)
    initial_params = [0.0, mean(z_f64)] # Simple initial guess
    try
        # Attempt to use the least squares solution as an initial guess
        # Check if matrix is square or vector is empty before attempting \
        if length(x_f64) >= 2
            A = hcat(x_f64, ones(length(x_f64))) # Create design matrix [x 1]
            initial_params_ls = A \ z_f64 # Solve Ax = b for x (parameters [a,
b])
            if length(initial_params_ls) == 2
                initial_params = initial_params_ls
            end
        end
    catch e
        println("Warning: Failed to compute least squares for initial guess.
Error: $e")
    end

    # Perform optimization
    result = Optim.optimize(sse, initial_params, Optim.LBFGS()) # Specify the
algorithm

    # Check convergence
    if !Optim.converged(result)
        @warn "Linear regression optimization did not converge!"
    end

    # Extract optimal parameters
    a, b = Optim.minimizer(result)
    return Float32(a), Float32(b) # Return Float32
end


"""
Processes a single laser scan: sorts by Z, takes the first N points,
performs linear regression, and returns a representative point.
"""
function calculate_laser_mins(laser_points::Vector{LaserRow}; num_points::Int =
20)::Vector{LaserRow}
    approximations::Vector{LaserRow} = []
    for row in laser_points
        n = length(row.x_array)

        # Sort by Z
        sort_idx = sortperm(row.z_array, rev = false)
        x_sorted = row.x_array[sort_idx]
        z_sorted = row.z_array[sort_idx]
        i_sorted = row.intensity_array[sort_idx]

        # Take the first num_points points
```

```
        limit = min(n, num_points)
        x_subset = x_sorted[1:limit]
        z_subset = z_sorted[1:limit]
        i_subset = i_sorted[1:limit]

        if limit < 2 # Need at least 2 points for regression
            @warn "Insufficient points ($limit) for regression in scan
$(row.time). Returning average/first point."
            # If there are few points, can return the average or the first
point
            if limit == 1
                # Use the single point
                return LaserRow(row.csv, row.tm, x_subset[1], z_subset[1],
i_subset[1])
            else # limit == 0 (although this case is handled above)
                return LaserRow(row.csv, row.tm, 0.0f0, 0.0f0, 0) # Or return
nothing?
            end
        end

        # Linear regression
        a, b = _linear_regression_optim(x_subset, z_subset)

        # Get the "average" point from regression
        x_mean = mean(x_subset)
        z_regr = a * x_mean + b
        i_val = i_subset[limit ÷ 2 + 1] # Median index (approximately)

        push!(approximations, LaserRow(row.csv, row.tm, [x_mean], [z_regr],
[i_val]))
    end
    return approximations
end


function calculate_points_on_grid(points::Vector{Point},
scans::Int64)::Vector{Height}
    if isempty(points)
        @warn "Input vector 'points' for interpolate_mins is empty.
Interpolation is not possible."
        return [] # Return an empty vector or handle otherwise
    end

    config = load_config()
    p0 = config.plane.p0
    p1 = config.plane.p1
    r_grid_max = sqrt((p1.y - p0.y)^2 + (p1.x - p0.x)^2)
    x_grid = p0.x:(Float64(p1.x) - p0.x) / scans:p1.x
    y_grid = p0.y:(Float64(p1.y) - p0.y) / scans:p1.y
    r_grid = 0.0:Float64(r_grid_max) / scans:r_grid_max
```

```
    # Extract scattered points
    x = [p.x for p in points]
    y = [p.y for p in points]
    z = [p.z for p in points]

    # Create points matrix (2xN for ScatteredInterpolation)
    # point_coords = hcat(x, y)' # Transpose to get 2 rows (x, y) and N columns
    # itp = ScatteredInterpolation.interpolate(Multiquadratic(), point_coords,
z) # Example: Nearest neighbor
    xy = []
    xy_grid = []
    if abs(p1.x - p0.x) < abs(p1.y - p0.y)
        xy = y
        xy_grid = y_grid
    else
        xy = x
        xy_grid = x_grid
    end

    xy_sorted = []
    z_sorted = []
    if !issorted(xy)
        # Получаем индексы для сортировки вектора xy_unsorted
        perm = sortperm(xy)

        # Применяем эту сортировку к обоим векторам
        xy_sorted = xy[perm]
        z_sorted = z[perm] # Важно применить ту же перестановку!
    else
        # Если уже отсортирован, просто используем как есть
        xy_sorted = xy
        z_sorted = z
    end

    # Теперь используем отсортированные данные для интерполяции
    # Эта строка должна заменить вашу строку 288 или использоваться после
подготовки данных
    itp = Interpolations.linear_interpolation(xy_sorted, z_sorted)

    # println(xy_grid)
    # println(z)
    # itp = Interpolations.linear_interpolation(xy, z)


    # Evaluate on the new grid
    heights::Vector{Height} = []
    # i = 1
    # for x_eval in x_grid
    #     z_evals = []
```

```
    #     for y_eval in y_grid
    #         # Evaluate expects points as columns (e.g., [x; y] or [x y]')
    #         z_eval = ScatteredInterpolation.evaluate(itp, [x_eval
y_eval]')[1] # Evaluate single point, get first result
    #         push!(z_evals, z_eval)
    #     end
    #     height = Height()
    #     height.x = x_eval
    #     height.y = y_grid[1]
    #     height.z = maximum(z_evals)
    #     height.r = r_grid[i]
    #     height.h = 0

    #     i += 1
    #     push!(heights, height)
    # end
    z_valid = z[1]
    for i in 1:length(x_grid)
        height = Height()
        # height.z = ScatteredInterpolation.evaluate(itp, [x_grid[i]
y_grid[i]]')[1]
        height.x = x_grid[i]
        height.y = y_grid[i]
        height.r = r_grid[i]
        try
            height.z = itp(xy_grid[i])
            z_valid = height.z
        catch
            height.z = z_valid
        end

        height.h = 0

        push!(heights, height)
    end
    return heights
end

function calculate_delta_heigts_destribution(points, output_path)

    # if layer_path empty (no layers1.csv)
    layer_path = joinpath(output_path, "layers")
    layer_path_current_height = joinpath(output_path, "current_height")

    layer_path_file = readdir(layer_path)

    if length(layer_path_file) == 0
        x = [p.x for p in points]
        y = [p.y for p in points]
        z = [p.z for p in points]
```

```
        r = [p.r for p in points]

        config = load_config()
        A = config.plane.A
        B = config.plane.B
        C = config.plane.C
        D = config.plane.D

        z_plane = (-A .* x .- B .* y .- D) ./ C
        h = abs.(A .* x .+ B .* y .+ C .* z .+ D) ./ sqrt(A^2 + B^2 + C^2)

        # h = z .- z_plane

        # if < 0 ---> 0
        h .= max.(h, 0.0) # Efficient way to clamp at 0

        # save to csv
        filename_csv = joinpath(layer_path, "layers1.csv")
        open(filename_csv, "w") do file
            write(file, "x,y,z,r,h\n") # Header
            for i in 1:length(x)
                write(file, string(x[i], ",", y[i], ",", z[i], ",", r[i], ",",
h[i], "\n"))
            end
        end

        return x, y, z, r, h
    else
        # load last csv layer layers*.csv
        layers = readdir(layer_path)
        layers = filter(x -> occursin(r"layers\d+\.csv", x), layers)
        layers = sort(layers, by = x -> parse(Int, match(r"\d+", x).match))
        last_layer = layers[end]

        # load last layer X,Y,Z,H
        last_layer_path = joinpath(layer_path, last_layer)
        x_last = []
        y_last = []
        z_last = []
        r_last = []
        h_last = []
        for line in eachline(last_layer_path)
            if line == "x,y,z,r,h"
                continue
            end
            data = split(line, ",")
            push!(x_last, parse(Float64, data[1]))
            push!(y_last, parse(Float64, data[2]))
            push!(z_last, parse(Float64, data[3]))
            push!(r_last, parse(Float64, data[4]))
```

```
            push!(h_last, parse(Float64, data[5]))
        end

        x = [p.x for p in points]
        y = [p.y for p in points]
        z = [p.z for p in points]
        r = [p.r for p in points]

        # h = z .- z_last
        h = sqrt.((x .- x_last).^2 + (y .- y_last).^2 + (z .- z_last).^2)

        # save to csv
        id = parse(Int, match(r"\d+", last_layer).match) + 1
        filename_csv = joinpath(layer_path, "layers$id.csv")
        open(filename_csv, "w") do file
            write(file, "x,y,z,r,h\n") # Header
            for i in 1:length(x)
                write(file, string(x[i], ",", y[i], ",", z[i], ",", r[i], ",",
h[i], "\n"))
            end
        end
        config = load_config()
        A = config.plane.A
        B = config.plane.B
        C = config.plane.C
        D = config.plane.D

        z_plane = (-A .* x .- B .* y .- D) ./ C
        h_current = z .- z_plane
        h_current .= max.(h_current, 0.0) # Efficient way to clamp at 0

        return x, y, z, r, h
    end

    return Nothing, Nothing, Nothing, Nothing, Nothing
end

function calculate_heigts_destribution(points, output_path)

    layer_path_current_height = joinpath(output_path, "current_height")

    layer_path_file = readdir(layer_path_current_height)

    last_layer = ""
    if length(layer_path_file) == 0
        last_layer = joinpath(layer_path_current_height, "layers1.csv")
    else
        layers = readdir(layer_path_current_height)
        layers = filter(x -> occursin(r"layers\d+\.csv", x), layers)
        layers = sort(layers, by = x -> parse(Int, match(r"\d+", x).match))
```

```
        last_layer = layers[end]
        id = parse(Int, match(r"\d+", last_layer).match) + 1
        last_layer = joinpath(layer_path_current_height, "layers$id.csv")
    end

    x = [p.x for p in points]
    y = [p.y for p in points]
    z = [p.z for p in points]
    r = [p.r for p in points]

    config = load_config()
    A = config.plane.A
    B = config.plane.B
    C = config.plane.C
    D = config.plane.D

    z_plane = (-A .* x .- B .* y .- D) ./ C

    # h = z .- z_plane
    h = abs.(A .* x .+ B .* y .+ C .* z .+ D) ./ sqrt(A^2 + B^2 + C^2)


    # if < 0 ---> 0
    h .= max.(h, 0.0) # Efficient way to clamp at 0

    # save to current_height.csv
    filename_csv = joinpath(layer_path_current_height, last_layer)
    open(filename_csv, "w") do file
        write(file, "x,y,z,r,h\n") # Header
        for i in 1:length(x)
            write(file, string(x[i], ",", y[i], ",", z[i], ",", r[i], ",",
h[i], "\n"))
        end
    end

    return x, y, z, r, h
end
```

**calc/plane.jl**

```
# calc/plane.jl
using LinearAlgebra
using Statistics

"""
    calculate_plane(points::Vector{Point}; num_points_edge::Int = 100) ->
Union{NTuple{4, Float64}, Nothing}
```

```
Calculates the parameters of the plane Ax + By + Cz + D = 0 that approximates
the point cloud,
using SVD on points from the edges of the (assumed sorted) cloud.
Returns a tuple (A, B, C, D), where the normal vector (A, B, C) is normalized.
Returns `nothing` if there are insufficient points or an error occurs.

# Arguments
- `points::Vector{Point}`: Vector of points in the cloud (assumed potentially
sorted by trajectory).
- `num_points_edge::Int`: Number of points to take from each edge for the
calculation.
"""
function calculate_plane(points::Vector{Point}; num_points_edge::Int =
100)::Union{NTuple{4, Float64}, Nothing}
    n_points = length(points)
    if n_points < 3
        @error "Insufficient points ($n_points) to define a plane."
        return nothing
    end

    # Select points from the edges
    # n_edge = min(num_points_edge, n_points ÷ 2)
    n_edge = n_points
    local points_for_plane::Vector{Point}
    if n_edge < 1 && n_points >= 3 # If the layer is short, but there are
enough points
        @warn "Layer is short, using all $n_points points to define the plane."
        points_for_plane = points
    elseif n_edge < 1 # If n_points < 3, already handled above
         @error "Insufficient points ($n_points) to define the plane (n_edge <
1)."
         return nothing
    else
        p_first = points[1:n_edge]
        # Ensure we don't take more points than available and indices are
correct
        start_last = max(1, n_points - n_edge + 1)
        p_last = points[start_last:end]
        points_for_plane = vcat(p_first, p_last)
        # Remove duplicates if ranges overlapped (for very short layers)
        unique!(points_for_plane)
        if length(points_for_plane) < 3
             @error "Insufficient unique points ($(length(points_for_plane)))
after edge selection to define the plane."
             return nothing
        end
    end

    # Convert points to a matrix (3 x N)
```

```
    points_matrix = Matrix{Float64}(undef, 3, length(points_for_plane))
    for (i, p) in enumerate(points_for_plane)
        points_matrix[:, i] = [p.x, p.y, p.z]
    end

    # Calculate the centroid of the selected points
    centroid = mean(points_matrix, dims=2) # 3x1 column vector

    # Center the points
    centered_points = points_matrix .- centroid

    try
        # Perform SVD
        U, _, _ = svd(centered_points)

        # The normal to the plane is the last left singular vector
        normal_vector = U[:, end]
        A, B, C = normal_vector[1], normal_vector[2], normal_vector[3]

        # Calculate D
        D = -dot(normal_vector, vec(centroid)) # vec() converts 3x1 to a vector

        # Check for vertical plane
         if abs(C) < 1e-9
            @warn "Found plane is almost vertical (C ≈ 0). Calculations might
be unstable."
         end

        # --- Assuming load_config/save_config handle a config object ---
        # --- This part interacts with external configuration ---
        conf = load_config() # Assumed function
        conf.plane.A = A
        conf.plane.B = B
        conf.plane.C = C
        conf.plane.D = D
        save_config(conf) # Assumed function
        # ----------------------------------------------------------

        return (A, B, C, D)

    catch e
        @error "Error calculating plane via SVD:" exception=(e,
catch_backtrace())
        return nothing
    end
end

"""
    filter_by_plane(points::Vector{Point}, plane_params::NTuple{4, Float64},
threshold::Float64) -> Vector{Point}
```

```
Filters points, keeping those that are within a specified *vertical*
distance (`threshold`) from the plane Ax + By + Cz + D = 0.

# Arguments
- `points::Vector{Point}`: Vector of original points.
# Returns
- `Vector{Point}`: Filtered vector of points.
"""
function filter_by_plane(points::Vector{Point})::Vector{Point}
    # --- Assuming load_config provides necessary parameters ---
    conf = Calc.load_config() # Assumed function/module
    A = conf.plane.A
    B = conf.plane.B
    C = conf.plane.C
    D = conf.plane.D
    threshold = conf.plane.StdZ # Assuming StdZ holds the threshold
    # ---------------------------------------------------------

    if abs(C) < 1e-9
        @error "Plane is almost vertical (C ≈ 0). Cannot calculate vertical
distance for filtering."
        # Return empty or original? Depends on requirements.
        return Point[] # Returning empty as filtering is not possible
    end

    filtered_points::Vector{Point} = []

    for p in points
        # z_plane = (-D - A * p.x - B * p.y) / C
        # vertical_distance = abs(p.z - z_plane)
        # if vertical_distance > threshold # Keep points FARTHER than the
threshold
        #     push!(filtered_points, p)
        # end
        distanc = abs(A*p.x + B*p.y + C*p.z + D) / sqrt(A^2 + B^2 + C^2)
        if distanc > threshold # Keep points FARTHER than the threshold
            push!(filtered_points, p)
        end
    end

    return filtered_points
end

function filter_laser_points(laser_rows::Vector{LaserRow})::Vector{LaserRow}
    config = load_config()
    laser_min = config.calibration.laser_min_x
    laser_max = config.calibration.laser_max_x

    new_laser_rows::Vector{LaserRow} = []
```

```
    for laser_row in laser_rows
        new_laser_row::LaserRow = LaserRow()
        new_laser_row.csv = laser_row.csv
        new_laser_row.tm = laser_row.tm
        for i in 1:length(laser_row.x_array)
            if laser_row.x_array[i] >= laser_min && laser_row.x_array[i] <=
laser_max
                push!(new_laser_row.x_array, laser_row.x_array[i])
                push!(new_laser_row.z_array, laser_row.z_array[i])
                push!(new_laser_row.intensity_array,
laser_row.intensity_array[i])
            end
        end
        push!(new_laser_rows, new_laser_row)
    end
    return new_laser_rows
end
```

**calc/points.jl**

```
# calc/points.jl
using Dates
using LinearAlgebra
using CSV
using DataFrames # Required by CSV.jl for reading/writing
using YAML       # Added for YAML parsing
using Base       # Ensure using Base: for basename

# ------------------------------------------------------ Structs
------------------------------------------------------------

"""
    LaserRow

Represents a single row of data from a laser scanner measurement, potentially
containing multiple points.
Assumes data comes from a CSV row like: TimeReceived,"X values","Z
values","Intensity values"
where values within quotes are space-separated.

# Fields
- `csv::String`: Source identifier (e.g., filename or description). *Note: Not
automatically populated during CSV read.*
- `tm::DateTime`: Timestamp of the measurement (from TimeReceived column).
- `x_array::Vector{Float64}`: Array of X-coordinates (space-separated in CSV).
- `z_array::Vector{Float64}`: Array of Z-coordinates (space-separated in CSV).
- `intensity_array::Vector{Int32}`: Array of intensity values (space-separated
```

```
integers in CSV).
"""
mutable struct LaserRow
    csv::String
    tm::DateTime
    x_array::Vector{Float64}
    z_array::Vector{Float64}
    intensity_array::Vector{Int32} # Changed to Int32

    # Default constructor for empty initialization - FIXED
    LaserRow() = new("", Dates.unix2datetime(0), Float64[], Float64[], Int32[])
    # Constructor with values
    LaserRow(csv, tm, x, z, intensity) = new(csv, tm, x, z, intensity)
end

"""
    RobotRow

Represents the pose (position and orientation) of a robot at a specific time.
Assumes data comes from a CSV row like: TimeReceived,X,Y,Z,RX,RY,RZ
Also used to represent calibration data loaded from YAML.

# Fields
- `csv::String`: Source identifier (e.g., filename or description). *Note: Not
automatically populated during CSV read.*
- `tm::DateTime`: Timestamp of the pose measurement (from TimeReceived column).
Defaults to epoch for calibration.
- `x::Float64`: X-coordinate of the robot's position or calibration offset.
- `y::Float64`: Y-coordinate of the robot's position or calibration offset.
- `z::Float64`: Z-coordinate of the robot's position or calibration offset.
- `rx::Float64`: Rotation around the X-axis (roll) in degrees (maps to 'a' in
YAML).
- `ry::Float64`: Rotation around the Y-axis (pitch) in degrees (maps to 'b' in
YAML).
- `rz::Float64`: Rotation around the Z-axis (yaw) in degrees (maps to 'c' in
YAML).
"""
mutable struct RobotRow
    csv::String
    tm::DateTime
    x::Float64
    y::Float64
    z::Float64
    rx::Float64 # Corresponds to 'a' in YAML calibration
    ry::Float64 # Corresponds to 'b' in YAML calibration
    rz::Float64 # Corresponds to 'c' in YAML calibration

    # Default constructor - FIXED
    RobotRow() = new("", Dates.unix2datetime(0), 0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
    # Constructor with values
```

```
    RobotRow(csv, tm, x, y, z, rx, ry, rz) = new(csv, tm, x, y, z, rx, ry, rz)
end

"""
    Point

Represents a single point in 3D space, typically calculated in the world frame.

# Fields
- `csv::String`: Source identifier (inherited from the LaserRow).
- `tm::DateTime`: Timestamp associated with the point's calculation.
- `x::Float64`: X-coordinate of the point.
- `y::Float64`: Y-coordinate of the point.
- `z::Float64`: Z-coordinate of the point.
"""
mutable struct Point
    csv::String
    tm::DateTime
    x::Float64
    y::Float64
    z::Float64

     # Default constructor - FIXED
     Point() = new("", Dates.unix2datetime(0), 0.0, 0.0, 0.0)
     # Constructor with values
     Point(csv, tm, x, y, z) = new(csv, tm, x, y, z)
end

# ------------------------------------------------------ Helper Functions
--------------------------------------------------

# Define common date formats - Primary format used by CSV.read and data_to_csv
const DATE_FORMAT_STR = "yyyy-mm-ddTHH:MM:SS.s"
# Secondary format to try if parsing fails (handles space separator)
const DATE_FORMAT_STR_SPACE = "yyyy-mm-dd HH:MM:SS.s"

# Create DateFormat objects for parsing performance
const DATE_FORMAT_T = Dates.DateFormat(DATE_FORMAT_STR)
const DATE_FORMAT_SPACE = Dates.DateFormat(DATE_FORMAT_STR_SPACE)

"""
    _parse_delimited_array(s::AbstractString, T::Type; delim=' ')

Internal helper function to parse a delimited string into a Vector{T}.
Returns an empty vector if parsing fails or the string is empty.

# Arguments
- `s::AbstractString`: The string to parse (e.g., "1.0 2.5 3.0").
- `T::Type`: The numeric type to parse into (e.g., Float64, Int32).
- `delim::Char`: The delimiter character (default is space).
```

```
# Returns
- `Vector{T}`: The parsed vector of numbers.
"""
function _parse_delimited_array(s::AbstractString, T::Type; delim='
')::Vector{T}
    s_stripped = strip(s) # Remove leading/trailing whitespace
    if isempty(s_stripped)
        return T[]
    end
    parts = split(s_stripped, delim; keepempty=false) # Don't keep empty
strings from multiple spaces
    result = Vector{T}(undef, length(parts))
    try
        for (i, part) in enumerate(parts)
            result[i] = parse(T, part)
        end
        return result
    catch e
        @warn "Failed to parse array string '$s' into Vector{$T} with delimiter
'$delim': $e"
        return T[] # Return empty on error
    end
end

"""
    _format_delimited_array(arr::Vector{<:Real}; delim=' ')

Internal helper function to format a vector of numbers into a delimited string.

# Arguments
- `arr::Vector{<:Real}`: The vector of numbers to format.
- `delim::Char`: The delimiter character (default is space).

# Returns
- `String`: The formatted string (e.g., "1.0 2.5 3.0").
"""
function _format_delimited_array(arr::Vector{<:Real}; delim=' ')::String
    return join(string.(arr), delim)
end

# ------------------------------------------------------ YAML Handling (New)
------------------------------------------------

"""
    get_calibration(filename::String="config.yaml") -> Union{RobotRow, Nothing}

Loads calibration data (x, y, z, a, b, c) from the 'calibration' section
of a YAML file and returns it as a `RobotRow` object.
Returns `nothing` if the file is not found, the section is missing,
```

```
keys are missing, or a parsing error occurs.

Note: 'a', 'b', 'c' from YAML are mapped to 'rx', 'ry', 'rz' in RobotRow.
The 'tm' field of the returned RobotRow defaults to the Unix epoch.
The 'csv' field is set to the filename.
"""
function get_calibration(filename::String="config.yaml")::Union{RobotRow,
Nothing}
    try
        if !isfile(filename)
            @error "Configuration file not found: $filename" # Translated from:
Файл конфигурации не найден
            return nothing
        end

        raw_config = YAML.load_file(filename)

        # Check for 'calibration' section
        if !haskey(raw_config, "calibration")
             @error "Missing required section 'calibration' in file $filename"
# Translated from: Отсутствует обязательная секция 'calibration'
             return nothing
        end
        cal_data = raw_config["calibration"]

        # Check for required keys within 'calibration'
        req_keys = ["x", "y", "z", "a", "b", "c"]
        for key in req_keys
            if !haskey(cal_data, key)
                @error "Missing required key '$key' in section 'calibration' of
file $filename" # Translated from: Отсутствует обязательный ключ
                return nothing
            end
        end

        # Create and populate RobotRow for calibration
        # Map YAML a, b, c to RobotRow rx, ry, rz
        calibration_row = RobotRow(
            "",                         # Source identifier
            Dates.unix2datetime(0),     # Default timestamp (epoch)
            Float64(cal_data["x"]),     # Convert just in case YAML parsed as Int
            Float64(cal_data["y"]),
            Float64(cal_data["z"]),
            Float64(cal_data["a"]),     # rx (roll)
            Float64(cal_data["b"]),     # ry (pitch)
            Float64(cal_data["c"])      # rz (yaw)
        )
        calibration_row.csv = filename # Set source identifier to the filename

        return calibration_row
```

```
    catch e
        @error "Error loading or parsing calibration from file '$filename':"
exception=(e, catch_backtrace()) # Translated from: Ошибка при загрузке или
парсинге калибровки из файла
        return nothing # Return nothing on any error
    end
end


# ------------------------------------------------------ CSV Handling
------------------------------------------------------

# --- LaserRow CSV ---

"""
    csv_to_data!(row::LaserRow, csv_row::DataFrameRow)

Populates a `LaserRow` struct from a `DataFrameRow`.
Assumes columns: :TimeReceived, :X, :Z, :Intensity
Where :X, :Z, :Intensity contain space-delimited strings.
Handles cases where :TimeReceived might be DateTime or String due to
inconsistent CSV formatting.
*Note: Does not populate the `row.csv` field.*

# Arguments
- `row::LaserRow`: The `LaserRow` struct to populate (mutated).
- `csv_row::DataFrameRow`: The row from the DataFrame.

# Returns
- `LaserRow`: The populated `row` object.
"""
function csv_to_data!(row::LaserRow, csv_row::DataFrameRow)::LaserRow
    try
        # --- Handle TimeReceived ---
        time_val = csv_row.TimeReceived
        if isa(time_val, DateTime)
            # Already parsed correctly by CSV.read
            row.tm = time_val
        elseif isa(time_val, AbstractString)
            # CSV.read failed to parse, try parsing manually with known formats
            time_str = string(time_val) # Ensure it's a standard String
            parsed = false
            try
                # Try parsing with the primary format ('T' separator)
                row.tm = DateTime(time_str, DATE_FORMAT_T)
                parsed = true
            catch e1
                try
                    # Try parsing with the secondary format (' ' separator)
```

```
                    row.tm = DateTime(time_str, DATE_FORMAT_SPACE)
                    parsed = true
                catch e2
                    @warn "Failed to parse TimeReceived string '$time_str' with
both formats: $e1 and $e2. Row data: $csv_row"
                    row.tm = Dates.unix2datetime(0) # Assign default on failure
                end
            end
        else
            # Unexpected type
             @warn "Unexpected type for TimeReceived: $(typeof(time_val)). Row
data: $csv_row"
             row.tm = Dates.unix2datetime(0) # Assign default
        end

        # --- Handle other fields ---
        # Parse space-delimited strings into arrays
        # Convert potential non-string types (like Missing) to string before
parsing arrays
        row.x_array = _parse_delimited_array(string(csv_row.X), Float64;
delim=' ')
        row.z_array = _parse_delimited_array(string(csv_row.Z), Float64;
delim=' ')
        row.intensity_array = _parse_delimited_array(string(csv_row.Intensity),
Int32; delim=' ') # Use Int32

    catch e
        @error "Error processing LaserRow from CSV: $e. Row data: $csv_row"
        # Print stacktrace for more debugging info
        showerror(stderr, e, catch_backtrace())
        println(stderr)
        # Reset row to default if parsing fails badly
        row.tm = Dates.unix2datetime(0)
        row.x_array = Float64[]
        row.z_array = Float64[]
        row.intensity_array = Int32[]
    end
    return row
end

"""
    data_to_csv(row::LaserRow)::NamedTuple

Converts a `LaserRow` struct into a `NamedTuple` suitable for writing to a CSV.
Outputs columns: TimeReceived, X, Z, Intensity with space-delimited values.
Uses the primary date format ('T' separator) for output consistency.
*Note: Does not include the `csv` field in the output.*

# Arguments
- `row::LaserRow`: The struct to convert.
```

```
# Returns
- `NamedTuple`: Data ready for CSV writing.
"""
function data_to_csv(row::LaserRow)::NamedTuple
    return (
        # Format output using the primary 'T' separator format string
        TimeReceived=Dates.format(row.tm, DATE_FORMAT_T),
        X=_format_delimited_array(row.x_array; delim=' '),
        Z=_format_delimited_array(row.z_array; delim=' '),
        Intensity=_format_delimited_array(row.intensity_array; delim=' ') #
Format Int32 array
    )
end

# --- RobotRow CSV ---

"""
    csv_to_data!(row::RobotRow, csv_row::DataFrameRow)

Populates a `RobotRow` struct from a `DataFrameRow`.
Assumes columns: :TimeReceived, :X, :Y, :Z, :RX, :RY, :RZ
Handles cases where :TimeReceived might be DateTime or String due to
inconsistent CSV formatting.
*Note: Does not populate the `row.csv` field.*

# Arguments
- `row::RobotRow`: The `RobotRow` struct to populate (mutated).
- `csv_row::DataFrameRow`: The row from the DataFrame.

# Returns
- `RobotRow`: The populated `row` object.
"""
function csv_to_data!(row::RobotRow, csv_row::DataFrameRow)::RobotRow
     try
        # --- Handle TimeReceived ---
        time_val = csv_row.TimeReceived
        if isa(time_val, DateTime)
            # Already parsed correctly by CSV.read
            row.tm = time_val
        elseif isa(time_val, AbstractString)
            # CSV.read failed to parse, try parsing manually with known formats
            time_str = string(time_val) # Ensure it's a standard String
            parsed = false
            try
                # Try parsing with the primary format ('T' separator)
                row.tm = DateTime(time_str, DATE_FORMAT_T)
                parsed = true
            catch e1
                try
```

```
                    # Try parsing with the secondary format (' ' separator)
                    row.tm = DateTime(time_str, DATE_FORMAT_SPACE)
                    parsed = true
                catch e2
                    @warn "Failed to parse TimeReceived string '$time_str' with
both formats: $e1 and $e2. Row data: $csv_row"
                    row.tm = Dates.unix2datetime(0) # Assign default on failure
                end
            end
        else
            # Unexpected type
             @warn "Unexpected type for TimeReceived: $(typeof(time_val)). Row
data: $csv_row"
             row.tm = Dates.unix2datetime(0) # Assign default
        end

        # --- Handle other fields ---
        # Parse numeric values
        # Handle potential Missing values by coalescing or checking ismissing
        # Use parse(Float64, string(value)) for robustness if CSV reads numbers
as strings
        # Use coalesce to provide "NaN" string if value is missing, then parse
        row.x = parse(Float64, string(coalesce(csv_row.X, "NaN")))
        row.y = parse(Float64, string(coalesce(csv_row.Y, "NaN")))
        row.z = parse(Float64, string(coalesce(csv_row.Z, "NaN")))
        row.rx = parse(Float64, string(coalesce(csv_row.RX, "NaN")))
        row.ry = parse(Float64, string(coalesce(csv_row.RY, "NaN")))
        row.rz = parse(Float64, string(coalesce(csv_row.RZ, "NaN")))
    catch e
        @error "Error processing RobotRow from CSV: $e. Row data: $csv_row"
        # Print stacktrace for more debugging info
        showerror(stderr, e, catch_backtrace())
        println(stderr)
        # Reset row to default if parsing fails badly
        row.tm = Dates.unix2datetime(0)
        row.x = row.y = row.z = row.rx = row.ry = row.rz = NaN # Use NaN for
invalid numeric data
    end
    return row
end

"""
    data_to_csv(row::RobotRow)::NamedTuple

Converts a `RobotRow` struct into a `NamedTuple` suitable for writing to a CSV.
Outputs columns: TimeReceived, X, Y, Z, RX, RY, RZ
Uses the primary date format ('T' separator) for output consistency.
*Note: Does not include the `csv` field in the output.*

# Arguments
```

```
- `row::RobotRow`: The struct to convert.

# Returns
- `NamedTuple`: Data ready for CSV writing.
"""
function data_to_csv(row::RobotRow)::NamedTuple
    return (
        # Format output using the primary 'T' separator format string
        TimeReceived=Dates.format(row.tm, DATE_FORMAT_T),
        X=row.x,
        Y=row.y,
        Z=row.z,
        RX=row.rx,
        RY=row.ry,
        RZ=row.rz
    )
end

# --- Generic CSV Reading ---

"""
    csv_to_data_array(::Type{T}, file::String)::Vector{T} where T <:
Union{RobotRow, LaserRow}

Reads a CSV file and converts its rows into a vector of the specified struct
type `T`
(either `RobotRow` or `LaserRow`). It dispatches to the correct `csv_to_data!`
method
based on the provided type `T`.
*Note: The `csv` field in the returned structs is not populated by this
function.*

# Arguments
- `::Type{T}`: The type of struct to create (e.g., `RobotRow` or `LaserRow`).
- `file::String`: Path to the CSV file.

# Returns
- `Vector{T}`: A vector containing the parsed data. Returns empty vector on
error.

# Example Usage
```julia
robot_data = csv_to_data_array(RobotRow, "robot_poses.csv")
laser_data = csv_to_data_array(LaserRow, "laser_scans.csv")
```
"""
function csv_to_data_array(::Type{T}, file::String)::Vector{T} where T <:
Union{RobotRow, LaserRow}
    result = T[] # Initialize vector of the specified type T
    local df # Ensure df is in local scope for the catch block if needed
```

```
    try
        # Read the CSV into a DataFrame.
        # Provide the primary dateformat. CSV.jl will attempt to parse using
this format.
        # If parsing fails for some rows due to format mismatch, they might
remain as strings.
        # Consider specifying `stringtype=String` if `InlineStrings` causes
issues.
        df = CSV.read(file, DataFrame; dateformat=DATE_FORMAT_STR) # Keep
primary format here

        # Iterate over DataFrame rows and convert each to the specified type T
        for csv_row in eachrow(df)
            row_instance = T() # Create a new instance of type T
            # csv_to_data! will now handle both DateTime and String types for
TimeReceived
            # and attempt to parse numeric fields robustly.
            push!(result, csv_to_data!(row_instance, csv_row))
        end
    catch e
        # Use correct type T in error message
        @error "Failed to read or process $(T) CSV '$file': $e"
        # Print stacktrace for more debugging info
        showerror(stderr, e, catch_backtrace())
        println(stderr) # Add a newline after stacktrace
        # Return empty vector on error
        return T[]
    end
    return result
end


# ------------------------------------------------- Transformation Logic
---------------------------------------------
# (No changes needed in this section, assuming it's correct)

"""
    euler_to_transform_y(data::RobotRow)::Matrix{Float64}

Converts Euler angles (ZYX convention, degrees) and translation from a
`RobotRow`
into a 4x4 homogeneous transformation matrix.

# Arguments
- `data::RobotRow`: Robot pose data containing translation (x, y, z) and
rotation (rx, ry, rz in degrees).

# Returns
- `Matrix{Float64}`: A 4x4 transformation matrix representing the robot's pose.
```

```
"""
function euler_to_transform_y(data::RobotRow)::Matrix{Float64}
    # Convert degrees to radians
    r_x = deg2rad(data.rx) # Roll
    r_y = deg2rad(data.ry) # Pitch
    r_z = deg2rad(data.rz) # Yaw

    # Calculate sines and cosines
    cos_rx, sin_rx = cos(r_x), sin(r_x)
    cos_ry, sin_ry = cos(r_y), sin(r_y)
    cos_rz, sin_rz = cos(r_z), sin(r_z)

    # Rotation matrix (ZYX order: Yaw, Pitch, Roll)
    # R = Rz(rz) * Ry(ry) * Rx(rx)
    result_matrix = zeros(Float64, 4, 4)

    # Column 1
    result_matrix[1, 1] = cos_ry * cos_rz
    result_matrix[2, 1] = cos_ry * sin_rz
    result_matrix[3, 1] = -sin_ry

    # Column 2
    result_matrix[1, 2] = sin_rx * sin_ry * cos_rz - cos_rx * sin_rz
    result_matrix[2, 2] = sin_rx * sin_ry * sin_rz + cos_rx * cos_rz
    result_matrix[3, 2] = sin_rx * cos_ry

    # Column 3
    result_matrix[1, 3] = cos_rx * sin_ry * cos_rz + sin_rx * sin_rz
    result_matrix[2, 3] = cos_rx * sin_ry * sin_rz - sin_rx * cos_rz
    result_matrix[3, 3] = cos_rx * cos_ry

    # Homogeneous coordinate
    result_matrix[4, 4] = 1.0

    # Fill the translation part (Column 4)
    result_matrix[1, 4] = data.x
    result_matrix[2, 4] = data.y
    result_matrix[3, 4] = data.z

    return result_matrix
end

"""
    transform_to_euler_y(input_matrix::Matrix{Float64})::RobotRow

Extracts Euler angles (ZYX convention, degrees) and translation from a 4x4
homogeneous transformation matrix and returns them in a `RobotRow`.

Handles gimbal lock singularity for pitch near +/- 90 degrees.
```

```
# Arguments
- `input_matrix::Matrix{Float64}`: A 4x4 transformation matrix.

# Returns
- `RobotRow`: A struct containing the extracted pose. `csv` and `tm` fields
will be empty/default.
"""
function transform_to_euler_y(input_matrix::Matrix{Float64})::RobotRow
    # Initialize with default values
    result = RobotRow() # Uses the default constructor

    # Extract translation
    result.x = input_matrix[1, 4]
    result.y = input_matrix[2, 4]
    result.z = input_matrix[3, 4]

    # Extract Euler angles (ZYX convention)
    # Pitch (ry)
    sin_pitch = -input_matrix[3, 1]
    # Clamp sin_pitch to avoid domain errors in asin due to potential floating
point inaccuracies
    if sin_pitch > 1.0
        sin_pitch = 1.0
    elseif sin_pitch < -1.0
        sin_pitch = -1.0
    end

    # Use a slightly more robust check for gimbal lock
    if abs(abs(sin_pitch) - 1.0) < 1e-9 # Check if sin_pitch is very close to
+1 or -1
        # Gimbal lock case
        result.ry = rad2deg(copysign(pi / 2.0, sin_pitch)) # +/- 90 degrees

        # Set roll (rx) to 0 and calculate yaw (rz) from atan2(R12, R22) -
common convention
        result.rz = rad2deg(atan(input_matrix[1, 2], input_matrix[2, 2])) # Yaw
        result.rx = 0.0 # Roll is arbitrary, set to 0
    else
        # Normal case
        result.ry = rad2deg(asin(sin_pitch)) # Pitch

        # Roll (rx) from atan2(R32, R33)
        result.rx = rad2deg(atan(input_matrix[3, 2], input_matrix[3, 3]))

        # Yaw (rz) from atan2(R21, R11)
        result.rz = rad2deg(atan(input_matrix[2, 1], input_matrix[1, 1]))
    end

    # Note: csv and tm fields remain at their default values (using
Dates.unix2datetime(0) now)
```

```
    # as they cannot be determined solely from the matrix.
    return result
end


"""
    calculate_points_row(robot_row::RobotRow, laser_row::LaserRow,
calibration::RobotRow)::Vector{Point}

Calculates the world coordinates of laser points for a single timestamp,
given the robot pose, laser data, and sensor calibration (offset).

# Arguments
- `robot_row::RobotRow`: The robot's pose at the time of the laser scan.
- `laser_row::LaserRow`: The laser scan data (points relative to the sensor).
- `calibration::RobotRow`: The pose of the laser sensor relative to the robot's
base frame.

# Returns
- `Vector{Point}`: A vector of calculated points in the world frame.
"""
function calculate_points_row(robot_row::RobotRow, laser_row::LaserRow,
calibration::RobotRow)::Vector{Point}
    # Calculate transformation from robot base to world
    T_world_robot = euler_to_transform_y(robot_row)
    # Calculate transformation from laser sensor to robot base
    T_robot_laser = euler_to_transform_y(calibration)

    # Combined transformation from laser sensor frame to world frame
    T_world_laser = T_world_robot * T_robot_laser

    num_points = min(length(laser_row.x_array), length(laser_row.z_array))
    # Intensity array length isn't used for coordinate calculation, but should
ideally match
    if num_points != length(laser_row.intensity_array)
        @warn "Mismatch between coordinate array lengths ($(num_points)) and
intensity array length ($(length(laser_row.intensity_array))) for timestamp
$(laser_row.tm) in source '$(laser_row.csv)'."
        # Decide how to handle: trim to shortest, error, or proceed?
        # Here, we proceed based on coordinate length.
        # Ensure num_points doesn't exceed intensity array length if
intensities are crucial later
        num_points = min(num_points, length(laser_row.intensity_array))
    end

    points = Vector{Point}(undef, num_points) # Preallocate vector

    for j in 1:num_points
        # Create homogeneous coordinate vector for the laser point (assuming
Y=0 in laser frame)
```

```
        # Point in laser frame: [lx, 0, lz, 1]
        p_laser = [laser_row.x_array[j], 0.0, laser_row.z_array[j], 1.0]

        # Transform point from laser frame to world frame
        p_world = T_world_laser * p_laser

        # Create Point struct with world coordinates and metadata
        # Extract world coordinates (first 3 elements of p_world)
        # Use timestamp and csv source from the *laser* data for the resulting
point
        # The .csv field from laser_row will likely be "" now.
        points[j] = Point(laser_row.csv, laser_row.tm, p_world[1], p_world[2],
p_world[3])
    end

    return points
end

"""
    calculate_points(robot_rows::Vector{RobotRow},
laser_rows::Vector{LaserRow}, calibration::RobotRow)::Vector{Point}

Calculates world coordinates for multiple laser scans based on corresponding
robot poses.
Assumes a one-to-one correspondence between robot_rows and laser_rows based on
index.
For time synchronization, a more robust approach (e.g., finding nearest
neighbors in time) might be needed.

# Arguments
- `robot_rows::Vector{RobotRow}`: A vector of robot poses.
- `laser_rows::Vector{LaserRow}`: A vector of laser scans.
- `calibration::RobotRow`: The pose of the laser sensor relative to the robot's
base frame.

# Returns
- `Vector{Point}`: A flattened vector containing all calculated points from all
scans.
"""
function calculate_points(robot_rows::Vector{RobotRow},
laser_rows::Vector{LaserRow}, calibration::RobotRow)::Vector{Point}
    if length(robot_rows) != length(laser_rows)
        @warn "Robot rows and Laser rows have different lengths
($(length(robot_rows)) vs $(length(laser_rows))). Processing only up to the
minimum length."
    end

    min_len = min(length(robot_rows), length(laser_rows))
    if min_len == 0
        return Point[] # Return empty if either input is empty
```

```
    end

    # Estimate total number of points for preallocation (can be rough)
    # Use average length of arrays in first few laser rows if available for
better estimate
    estimated_points_per_row = isempty(laser_rows) ||
isempty(laser_rows[1].x_array) ? 50 : length(laser_rows[1].x_array) # Guess 50
if first is empty
    estimated_points = min_len * estimated_points_per_row

    points = Vector{Point}(undef, 0) # Initialize empty
    sizehint!(points, estimated_points) # Preallocate roughly

    for i in 1:min_len
        # Check if timestamps are reasonably close (optional, but good
practice)
        # Ensure both tm fields are valid DateTimes before comparing
        if isa(robot_rows[i].tm, DateTime) && isa(laser_rows[i].tm, DateTime)
&&
           robot_rows[i].tm != Dates.unix2datetime(0) && laser_rows[i].tm !=
Dates.unix2datetime(0) # Check not default
            time_diff = abs(robot_rows[i].tm - laser_rows[i].tm)
            if time_diff > Millisecond(500) # Example threshold: 500ms
                @warn "Timestamp mismatch for index $i:
Robot=$(robot_rows[i].tm) ('$(robot_rows[i].csv)'), Laser=$(laser_rows[i].tm)
('$(laser_rows[i].csv)'). Difference: $time_diff"
            end
        else
             @warn "Skipping timestamp comparison for index $i due to invalid
or default DateTime."
        end


        # Check if laser row data is valid before processing
        if isempty(laser_rows[i].x_array) || isempty(laser_rows[i].z_array)
             @warn "Skipping index $i due to empty coordinate arrays in
LaserRow: Timestamp=$(laser_rows[i].tm), Source='$(laser_rows[i].csv)'"
             continue # Skip to the next iteration
        end

        # Check if robot row data is valid (e.g., not default values from
parsing error)
        if robot_rows[i].tm == Dates.unix2datetime(0) || isnan(robot_rows[i].x)
# Check timestamp and a numeric field
             @warn "Skipping index $i due to invalid RobotRow data (default
timestamp or NaN): $(robot_rows[i])"
             continue
        end
```

```
        points_row = calculate_points_row(robot_rows[i], laser_rows[i],
calibration)
        append!(points, points_row) # Append the vector of points from the row
    end
    return points
end
```

**plot/plot.jl**

```
# plot/plot.jl
module Plot

using Makie, GLMakie
using Colors
using LinearAlgebra
using Printf # Added for formatted coordinate strings

function primary_resolution()
    monitor = GLMakie.GLFW.GetPrimaryMonitor()
    videomode = GLMakie.MonitorProperties(monitor).videomode
    return (videomode.width, videomode.height)
end

# Create Figure and Axes
HLC_FIG = Figure(size = primary_resolution())
HLC_AX_HD = Axis(HLC_FIG[1, 1],
    title="Delta Heights Distribution",
    xlabel = "step (mm)", ylabel = "Δheight (mm)",
    aspect = DataAspect()
)

HLC_AX_CHD = Axis(HLC_FIG[2, 1],
    title="Heights Distribution",
    xlabel = "step (mm)", ylabel = "height (mm)",
    aspect = DataAspect()
)

HLC_AX_PC = Axis3(HLC_FIG[3, 1],
    title="Point Cloud",
    xlabel = "x (mm)", ylabel = "y (mm)", zlabel = "z (mm)",
    aspect=:data,
    viewmode = :free,
    zoommode = :fit
)

# --- Глобальная ссылка для хранения объекта текущей легенды HLC_AX_HD ---
# Инициализируем `nothing`, так как сначала легенды нет.
```

```
# Используем Ref для хранения изменяемой ссылки.
HLC_AX_HD_LEGEND = Ref{Union{Legend, Nothing}}(nothing)
HLC_AX_CHD_LEGEND = Ref{Union{Legend, Nothing}}(nothing)
HLC_AX_PC_LEGEND = Ref{Union{Legend, Nothing}}(nothing)
# ------------------------------------------------------------------

# Global variables for layer/dataset IDs
HLC_AX_HD_ID = 0 # For Delta Heights
HLC_AX_CHD_ID = 0 # For Heights
HLC_AX_PC_ID = 0  # For Point Cloud

HLC_INSPECTOR = DataInspector(HLC_FIG)

"""
    update_delta_heigts_destribution(r, h)

Updates the Delta Heights Distribution plot (HLC_AX_HD).
Explicitly deletes the previous legend before creating and assigning a new one.
Labels are generated as "Layer N to N+1".

# Arguments
- `r`: Vector of radius values.
- `h`: Vector of height values.
"""
function update_delta_heigts_destribution(r, h)
    global HLC_AX_HD_ID       # Necessary to modify the global variable
    global HLC_AX_HD_LEGEND  # Necessary to access and modify the legend
reference

    current_id = HLC_AX_HD_ID
    HLC_AX_HD_ID += 1 # Increment ID for the next call
    current_label = "Layer $(current_id) to $(current_id + 1)"

    scatter!(HLC_AX_HD, r, h, markersize=5, label=current_label)

    # --- Шаг 1: Удаление предыдущей легенды (если она существует) ---
    # Проверяем, есть ли ссылка на легенду и не была ли она уже удалена
    if !isnothing(HLC_AX_HD_LEGEND[]) && HLC_AX_HD_LEGEND[].parent !== nothing
        try
            delete!(HLC_AX_HD_LEGEND[]) # Удаляем объект легенды
            HLC_AX_HD_LEGEND[] = nothing # Сбрасываем ссылку после успешного
удаления
        catch e
            @warn "Could not delete previous legend object: $e"
            # Все равно сбрасываем ссылку, чтобы избежать повторных попыток
удаления
            HLC_AX_HD_LEGEND[] = nothing
        end
    end
    # ------------------------------------------------------------------
```

```
    new_legend = Legend(HLC_FIG, HLC_AX_HD, "ΔLayers") # Создаем новый объект
    HLC_FIG[1, 2] = new_legend # Помещаем новую легенду в ячейку [1, 2]
    HLC_AX_HD_LEGEND[] = new_legend # Обновляем глобальную ссылку

    # Display the Figure
    display(HLC_FIG)
end
"""
    update_heigts_destribution(r, h)

Updates the Heights Distribution plot (HLC_AX_CHD).
Labels are generated as "Layer N".

# Arguments
- `r`: Vector of radius values.
- `h`: Vector of height values.
"""
function update_heigts_destribution(r, h)
    global HLC_AX_CHD_ID # Necessary to modify the global variable
    global HLC_AX_CHD_LEGEND
    HLC_AX_CHD_ID += 1 # Increment ID before use (start with Layer 1)
    current_label = "Layer $(HLC_AX_CHD_ID)"
    scatter!(HLC_AX_CHD, r, h, markersize=5, label=current_label)
    # Consider adding a legend update here too, similar to the delta heights
plot, if desired
    # Example:
    # HLC_FIG[2, 2] = Legend(HLC_FIG, HLC_AX_CHD, "Layers")
    if !isnothing(HLC_AX_CHD_LEGEND[]) && HLC_AX_CHD_LEGEND[].parent !==
nothing
        try
            delete!(HLC_AX_CHD_LEGEND[]) # Удаляем объект легенды
            HLC_AX_CHD_LEGEND[] = nothing # Сбрасываем ссылку после успешного
удаления
        catch e
            @warn "Could not delete previous legend object: $e"
            # Все равно сбрасываем ссылку, чтобы избежать повторных попыток
удаления
            HLC_AX_CHD_LEGEND[] = nothing
        end
    end
    new_legend = Legend(HLC_FIG, HLC_AX_CHD, "Layers") # Создаем новый объект
    HLC_FIG[2, 2] = new_legend # Помещаем новую легенду в ячейку [1, 2]
    HLC_AX_CHD_LEGEND[] = new_legend # Обновляем глобальную ссылку

    display(HLC_FIG)
end

"""
    update_cloud_points(points)
```

```
Updates the 3D point cloud plot (HLC_AX_PC).
Labels are generated as "Points N".

# Arguments
- `points`: Vector of points with `.x`, `.y`, `.z` fields.
"""
function update_cloud_points(points)
    global HLC_AX_PC_ID # Necessary to modify the global variable
    global HLC_AX_PC_LEGEND
    # Check if the points array is empty
    if isempty(points)
        @warn "The 'points' array is empty in the update_cloud_points function"
        return
    end

    HLC_AX_PC_ID += 1 # Increment ID before use (start with Points 1)
    current_label = "Points $(HLC_AX_PC_ID)"

    # Extract x, y, z coordinates
    x = [p.x for p in points]
    y = [p.y for p in points]
    z = [p.z for p in points]

    # Add points to the 3D Axis
    scatter!(HLC_AX_PC, x, y, z, markersize=1, label=current_label)

    # Consider adding a legend update here too, similar to the delta heights
plot, if desired
    # Example:
    # HLC_FIG[3, 2] = Legend(HLC_FIG, HLC_AX_PC, "Layers")
    if !isnothing(HLC_AX_PC_LEGEND[]) && HLC_AX_PC_LEGEND[].parent !== nothing
        try
            delete!(HLC_AX_PC_LEGEND[]) # Удаляем объект легенды
            HLC_AX_PC_LEGEND[] = nothing # Сбрасываем ссылку после успешного
удаления
        catch e
            @warn "Could not delete previous legend object: $e"
            # Все равно сбрасываем ссылку, чтобы избежать повторных попыток
удаления
            HLC_AX_PC_LEGEND[] = nothing
        end
    end
    new_legend = Legend(HLC_FIG, HLC_AX_PC, "Points") # Создаем новый объект
    HLC_FIG[3, 2] = new_legend # Помещаем новую легенду в ячейку [1, 2]
    HLC_AX_PC_LEGEND[] = new_legend # Обновляем глобальную ссылку

    # Display the Figure
    display(HLC_FIG)
end
```

```
"""
    update_plane(points)

Updates the 3D point cloud plot (HLC_AX_PC).
Labels are generated as "Points N".

# Arguments
- `points`: Vector of points with `.x`, `.y`, `.z` fields.
"""
function update_plane(points)
    global HLC_AX_PC_ID # Necessary to modify the global variable
    global HLC_AX_PC_LEGEND
    # Check if the points array is empty
    if isempty(points)
        @warn "The 'points' array is empty in the update_cloud_points function"
        return
    end

    HLC_AX_PC_ID += 1 # Increment ID before use (start with Points 1)
    current_label = "Points $(HLC_AX_PC_ID)"

    # Extract x, y, z coordinates
    x = [p.x for p in points]
    y = [p.y for p in points]
    z = [p.z for p in points]

    # Add points to the 3D Axis
    scatter!(HLC_AX_PC, x, y, z, markersize=1, color=:red, label=current_label)

    # Consider adding a legend update here too, similar to the delta heights
plot, if desired
    # Example:
    # HLC_FIG[3, 2] = Legend(HLC_FIG, HLC_AX_PC, "Layers")
    if !isnothing(HLC_AX_PC_LEGEND[]) && HLC_AX_PC_LEGEND[].parent !== nothing
        try
            delete!(HLC_AX_PC_LEGEND[]) # Удаляем объект легенды
            HLC_AX_PC_LEGEND[] = nothing # Сбрасываем ссылку после успешного
удаления
        catch e
            @warn "Could not delete previous legend object: $e"
            # Все равно сбрасываем ссылку, чтобы избежать повторных попыток
удаления
            HLC_AX_PC_LEGEND[] = nothing
        end
    end
    new_legend = Legend(HLC_FIG, HLC_AX_PC, "Points") # Создаем новый объект
    HLC_FIG[3, 2] = new_legend # Помещаем новую легенду в ячейку [1, 2]
    HLC_AX_PC_LEGEND[] = new_legend # Обновляем глобальную ссылку

    # Display the Figure
```

```
    display(HLC_FIG)
end

end # --- End of your Plot module ---
```

**hlc.jl**

```
# hlc.jl
module HLC

# ---------------------------------------- Dependencies
-----------------------------------------------------
using NativeFileDialog
using Dates

include("calc/calc.jl")
include("plot/plot.jl")

# ---------------------------------------- Structures / Functions
----------------------------------------
"""
    monitor_path(path::String)

Placeholder function for monitoring a directory.
"""
function monitor_path(path::String, interval::Int64)
    println("Starting path monitoring: ", path)
    # Add your actual monitoring logic here
    # For example, starting FileWatching.watch_folder or another mechanism
    # laser_rows = Calc.csv_to_data_array(Calc.LaserRow, joinpath(path,
"LaserCoordinate1.csv"))
    # robot_rows = Calc.csv_to_data_array(Calc.RobotRow, joinpath(path,
"RobotCoordinate1.csv"))
    # cal = Calc.get_calibration()

    # point_cloud = Calc.calculate_points(robot_rows, laser_rows, cal)

    # Plot.update_cloud_points(point_cloud)

    # clean layer_path if it exists
    if isdir(joinpath(path, "layers"))
        rm(joinpath(path, "layers"), recursive=true)
    end
    if isdir(joinpath(path, "current_height"))
        rm(joinpath(path, "current_height"), recursive=true)
    end

    mkdir(joinpath(path, "layers"))
```

```
    mkdir(joinpath(path, "current_height"))

    monitored_dir = abspath(path)
    println("="^40)
    @info "Starting directory monitoring: $monitored_dir"
    println("="^40)

    last_confirmed_index::Int = 0 # The last index for which BOTH files were
found

    try
         if isdir(monitored_dir)
             current_index = 1
             while true
                 robot_file = "RobotCoordinate$(current_index).csv"
                 laser_file = "LaserCoordinate$(current_index).csv"
                 robot_path = joinpath(monitored_dir, robot_file)
                 laser_path = joinpath(monitored_dir, laser_file)

                 if isfile(robot_path) && isfile(laser_path)
                     # Both files for this index exist
                     last_confirmed_index = current_index
                     @debug "Found existing confirmed index:
$last_confirmed_index"
                     current_index += 1 # Check the next one
                 else
                     # The pair is incomplete or missing, stop checking
                     @info "Initial confirmed index: $last_confirmed_index.
Wait scans files..."
                     break
                 end
             end
         else
             @warn "Target directory '$monitored_dir' not found during initial
check."
             # last_confirmed_index remains 0
         end
    catch e
        @error "Error during initial index check in '$monitored_dir': "
exception=(e, catch_backtrace())
        # Can decide to exit or continue with last_confirmed_index = 0
    end


    # --- Main monitoring loop ---
    try
        conf = Calc.load_config()
        calibration = Calc.RobotRow(
            "", Dates.unix2datetime(0), # empty lines
            conf.calibration.x, conf.calibration.y, conf.calibration.z, # x, y,
```

```
z
            conf.calibration.a, conf.calibration.b, conf.calibration.c, # rx,
ry, rz - same parametrs as a, b, c
        )
        while true
            # Check directory availability (in case it's deleted during
operation)
            if !isdir(monitored_dir)
                @error "Directory '$monitored_dir' is no longer available!
Stopping monitoring."
                break # Break the while loop
            end

            next_index_to_check = last_confirmed_index + 1
            @debug "Checking index: $next_index_to_check"

            robot_file_next = "RobotCoordinate$(next_index_to_check).csv"
            laser_file_next = "LaserCoordinate$(next_index_to_check).csv"

            robot_path_next = joinpath(monitored_dir, robot_file_next)
            laser_path_next = joinpath(monitored_dir, laser_file_next)

            # Check for the existence of BOTH files for the next index
            if isfile(robot_path_next) && isfile(laser_path_next)
                # --- New pair found ---
                @info "Detected complete pair for index: $next_index_to_check"

                # Report readiness of the PREVIOUS index (if not zero)
                if last_confirmed_index > 0
                    prev_index = last_confirmed_index
                    robot_file_prev = joinpath(monitored_dir,
"RobotCoordinate$(prev_index).csv")
                    laser_file_prev = joinpath(monitored_dir,
"LaserCoordinate$(prev_index).csv")

                    # read laser and robot points
                    robot_rows = Calc.csv_to_data_array(Calc.RobotRow,
robot_file_prev)
                    laser_rows = Calc.csv_to_data_array(Calc.LaserRow,
laser_file_prev)
                    laser_rows = Calc.filter_laser_points(laser_rows)

                    # calculate points
                    cloud_points = Calc.calculate_points(robot_rows,
laser_rows, calibration)

                    # firts scan - plane
                    if prev_index == 1
                        # calculate plain and save to config.yaml
                        plane_params = Calc.calculate_plane(cloud_points)
```

```
                        # update plots (with plane)
                        Plot.update_plane(cloud_points)
                    else
                        # find max approximated
                        laser_rows_mins = Calc.calculate_laser_mins(laser_rows)

                        # calculate points and filter it by plane
                        cloud_points_mins = Calc.calculate_points(robot_rows,
laser_rows_mins, calibration)
                        cloud_points_mins =
Calc.filter_by_plane(cloud_points_mins)

                        # interpolate it, create grid
                        if prev_index == 2
                            Calc.calculate_grid_range(cloud_points_mins)
                        end
                        heights_on_grid::Vector{Calc.Height} =
Calc.calculate_points_on_grid(cloud_points_mins, conf.plane.scans)

                        # create delta heights description and current heights
description
                        x_delta, y_delta, z_delta, r_delta, h_delta =
Calc.calculate_delta_heigts_destribution(heights_on_grid, monitored_dir)
                        x, y, z, r, h =
Calc.calculate_heigts_destribution(heights_on_grid, monitored_dir)

                        # plot it
                        Plot.update_delta_heigts_destribution(r_delta, h_delta)
                        Plot.update_heigts_destribution(r, h)
                        Plot.update_cloud_points(cloud_points)
                    end

                elseif next_index_to_check == 1 # Special case for the first
found index
                    @info "This is the first detected pair (index 1). No
previous files."
                end

                # Update state
                last_confirmed_index = next_index_to_check

                # DO NOT wait for the interval, immediately check the next
index,
                # as multiple pairs might have appeared simultaneously
                @info "Scan with index: $next_index_to_check was processed!"
                continue

            else
                # Pair for next_index_to_check is not ready yet (or doesn't
```

```
exist)
                @debug "Pair for index $next_index_to_check not found.
Waiting..."
                # Wait before the next check
                sleep(interval)
            end

        end # while true

    catch e
        # Handle interruption (Ctrl+C)
        if e isa InterruptException
            println("\n\n[", now(), "] Interrupt signal received (Ctrl+C).
Shutting down...")
        else
            @error "Unexpected error in monitoring loop:" exception=(e,
catch_backtrace())
            rethrow(e) # Rethrow the error for diagnostics
        end
    finally
        println("[", now(), "] Monitoring stopped.")
    end
end



"""
    run()

Starts the HLC process. Opens a graphical dialog for the user
to select a folder and then initiates monitoring of that folder.
"""
function run()
    println("Waiting for user to select a folder via GUI...")

    # Open the native folder selection dialog
    path = pick_folder() # Returns the selected path or "" if the user
cancelled

    # Check if a path was selected (user didn't press "Cancel")
    if isempty(path)
        println("Folder selection cancelled by user.")
        return # Exit if no folder was selected
    end

    # Optionally: Check if the selected path is actually a folder
    if !isdir(path)
        # Ideally, show a graphical error message here too,
        # but for simplicity, keep console output for now.
        @error "Selected path is not a folder or is inaccessible: $path"
```

```
        # Could use `run(\`cmd /c msg * "Error text"\`)` on Windows
        # or `run(\`zenity --error --text="Error text"\`)` on Linux (if zenity
is installed)
        # for a simple graphical notification.
        return # Exit if it's not a folder
    end

    println("Folder selected: ", path)
    println("Starting monitoring...")
    #
monitor_path("C:\\Users\\Admin\\Projects\\Additive-Lab-Software\\CourceProject3
\\HLC\\test\\2025-03-20_14-52-45", 2) # Call the monitoring function with the
selected path
    monitor_path(path, 2)
end

end # end module HLC
```

**main.jl**

```
# main.jl
include("hlc.jl")

HLC.run()
```

**config.yaml**

```
calibration:
  x: 156.84264
  y: -14.302736
  z: 415.915983
  a: -66.445139
  b: -1.102508
  c: 88.889557
  laser_min_x: -10.0
  laser_max_x: 10.0
plane:
  exist: false
  A: 0.005509066936502152
  B: 0.020559412146754323
  C: 0.9997734547154516
  D: 62.160391941973934
  StdZ: 2.0
  p0:
    x: 1525.6862388102713
```

```
  y: -72.01546378345972
p1:
  x: 1387.0763174911701
  y: -70.6212648255251
scans: 500
```